

OBJECT-ORIENTED PROGRAMMING

A Complete Interview Guide

From Fundamentals to Design Patterns to LLD Interview Prep

For Product / Service / Startup Interviews

- Chapter 1 Introduction to OOP
- Chapter 2 Classes and Objects
- Chapter 3 Encapsulation
- Chapter 4 Inheritance
- Chapter 5 Polymorphism
- Chapter 6 Abstraction
- Chapter 7 SOLID Principles
- Chapter 8 Design Patterns
- Chapter 9 Advanced OOP Concepts
- Chapter 10 Interview Patterns & Common Qs

What's inside

- Four pillars from scratch • UML diagrams for every concept
- SOLID with concrete examples • 15+ design patterns • Language gotchas (Java, C++, Python)
- LLD interview approach template • 10 most-asked LLD problems mapped out

Table of Contents

Chapter 1: Introduction to OOP

- 1.1 What is a Programming Paradigm?
- 1.2 Why OOP Exists
- 1.3 The Core Idea: Objects and Classes
- 1.4 The Four Pillars of OOP
- 1.5 Benefits and Drawbacks of OOP

Chapter 2: Classes and Objects

- 2.1 Anatomy of a Class
- 2.2 From Class to Object — Instantiation
- 2.3 Constructors (default, parameterized, copy, chaining)
- 2.4 Destructors and Object Lifecycle
- 2.5 The this / self Reference
- 2.6 Static Members

Chapter 3: Encapsulation

- 3.1 What is Encapsulation?
- 3.2 Why Encapsulate?
- 3.3 Access Modifiers (private, protected, public)
- 3.4 Getters and Setters (done right)
- 3.5 Immutability

Chapter 4: Inheritance

- 4.1 What is Inheritance?
- 4.2 Types of Inheritance
- 4.3 What Gets Inherited?
- 4.4 Method Overriding
- 4.5 Composition vs Inheritance — The Big Debate
- 4.6 Multiple Inheritance and the Diamond Problem
- 4.7 Preventing Inheritance (final, sealed)

Chapter 5: Polymorphism

- 5.1 What is Polymorphism?
- 5.2 Method Overloading (Compile-Time)
- 5.3 Method Overriding (Runtime)
- 5.4 Dynamic Dispatch and vtables
- 5.5 Upcasting and Downcasting

Chapter 6: Abstraction

- 6.1 What is Abstraction?
- 6.2 Abstract Classes
- 6.3 Interfaces (classic and modern)
- 6.4 Abstract Class vs Interface — The Interview Classic

Chapter 7: SOLID Principles

- 7.1 Overview
- 7.2 S — Single Responsibility
- 7.3 O — Open / Closed
- 7.4 L — Liskov Substitution
- 7.5 I — Interface Segregation
- 7.6 D — Dependency Inversion

Chapter 8: Design Patterns

- 8.1 What are Design Patterns?
- 8.2-8.5 Creational (Singleton, Factory, Builder, Prototype)
- 8.6-8.10 Structural (Adapter, Decorator, Facade, Proxy, Composite)
- 8.11-8.17 Behavioral (Strategy, Observer, Command, Template, Iterator, State, CoR)

Chapter 9: Advanced OOP Concepts

- 9.1 Coupling and Cohesion
- 9.2 Object Memory — Stack vs Heap
- 9.3 Shallow Copy vs Deep Copy
- 9.4 Object Equality vs Identity (equals/hashCode contract)
- 9.5 Generics and Templates (type erasure)
- 9.6 Reflection and Metaprogramming
- 9.7 MRO and Virtual Inheritance
- 9.8 Composition Deep Dive (Aggregation vs Composition)

Chapter 10: Interview Patterns & Common Questions

- 10.1 The OOP Interview Landscape
- 10.2 Most-Asked Conceptual Questions with Ideal Answers
- 10.3 Code Smells Interviewers Look For
- 10.4 Language-Specific Gotchas (Java, C++, Python)
- 10.5 The LLD Interview — Template Approach
- 10.6 The Most-Asked LLD Problems (Parking Lot, ATM, LRU, etc.)
- 10.7 Final Interview Cheat Sheet

Appendix: Quick Reference — 4 Pillars, SOLID, 23 GoF Patterns

Chapter 1

Introduction to Object-Oriented Programming

OOP isn't just syntax — it's a way of thinking. Before we dive into classes, inheritance, and design patterns, let's understand what OOP is trying to solve and why it dominates modern software.

1.1 What is a Programming Paradigm?

A **programming paradigm** is a fundamental style of building the structure and elements of computer programs. It's how you organize your thinking about the problem — not just what syntax you type. Major paradigms include:

- **Procedural:** A program is a sequence of procedures (functions) operating on data. C, Pascal, early BASIC.
- **Object-Oriented:** A program is a collection of interacting objects, each combining data and behavior. Java, C++, C#, Python, Ruby, Swift.
- **Functional:** A program is a composition of pure functions with immutable data. Haskell, Lisp, Erlang, Elixir; increasingly influencing all others.
- **Declarative:** Describe what you want, not how to compute it. SQL, HTML, Prolog.

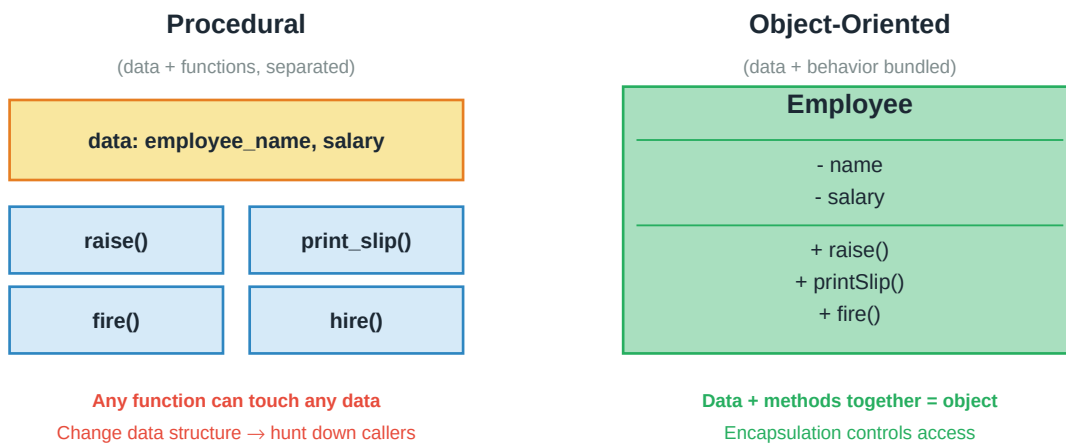
Most modern languages are multi-paradigm — Python, JavaScript, and Kotlin freely mix OOP with functional and procedural styles. But OOP is the default way most enterprise software is structured.

1.2 Why OOP Exists

In the 1960s, programs were procedural. Data was global; functions worked on it. This worked for small programs, but as programs grew, three problems emerged:

- **Complexity explosion.** Every function could touch any data. Understanding the codebase meant understanding everything at once.
- **Fragile changes.** Changing a data structure required updating dozens of callers scattered everywhere. Miss one, and you got subtle bugs.
- **Poor modeling of real world.** Real systems have things (customers, orders, accounts) that have both state and behavior. Splitting them into functions and structs felt artificial.

Procedural vs Object-Oriented Thinking



OOP shifts the mental model: 'what things exist and what can they do'

instead of 'what steps do I need to perform in what order'

Figure: Procedural vs OOP — the fundamental shift in how we organize code

OOP was invented (Simula 67, Smalltalk 72) to solve these by bundling data with the operations that manipulate it. The world became a network of objects sending messages to each other.

Analogy: Think of a company. In procedural thinking, everyone shares one big filing cabinet — anyone can grab any file, and functions like `calculate_salary()` reach into it. In OOP thinking, each employee has their own folder — with their own logic to compute their salary, print their slip, etc. Others can ASK the employee for information but can't rummage in their folder.

1.3 The Core Idea: Objects and Classes

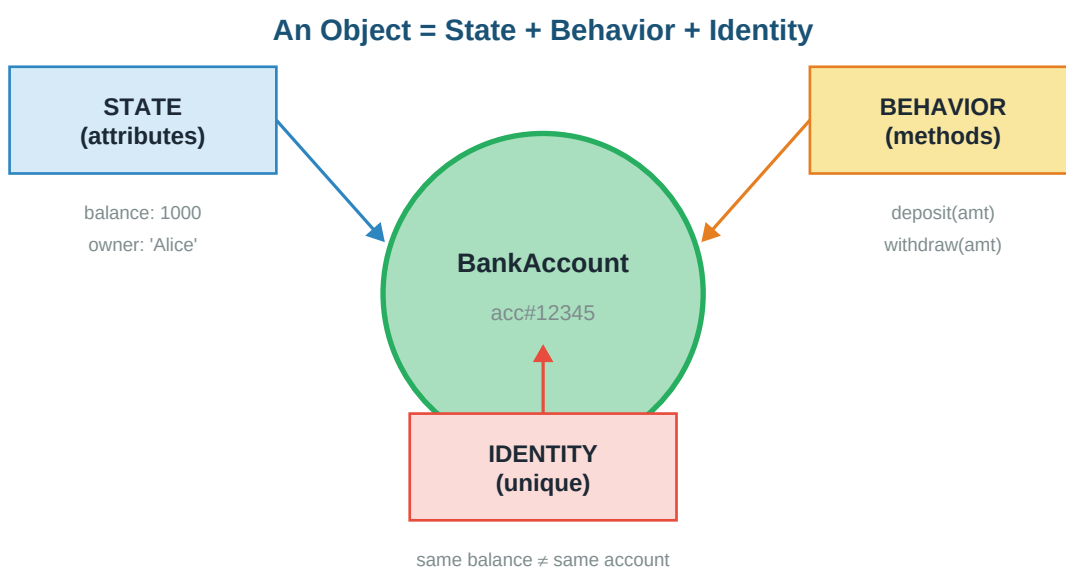


Figure: An object bundles state, behavior, and identity

An **object** is a self-contained unit that has three properties:

- **State (attributes):** Data the object holds. A BankAccount has a balance, an owner, an account number.
- **Behavior (methods):** Operations the object can perform. A BankAccount can deposit, withdraw, check balance.
- **Identity:** A unique existence, distinct from any other object — even one with identical state. Two accounts with the same balance are still two different accounts.

A **class** is a blueprint that describes what state and behaviors instances (objects) will have. One class → many objects, each with independent state but same structure and behavior definitions.

In many languages, classes are themselves "objects" too — you can pass them around, inspect them at runtime, and even modify them. But conceptually: class = category, object = instance of that category.

1.4 The Four Pillars of OOP

OOP rests on four fundamental concepts, sometimes called "the four pillars." You'll see them everywhere — in interviews, textbooks, and every OOP language:

- **Encapsulation:** Bundle data and methods together; hide internals behind a public interface. Chapter 3.
- **Inheritance:** Reuse and extend behavior of an existing class. A specialized class inherits from a general one. Chapter 4.
- **Polymorphism:** One interface, many implementations. The same call behaves differently depending on the actual object type. Chapter 5.
- **Abstraction:** Hide complex implementation, expose only what's necessary. Chapter 6.

Key Insight: For interviews: don't just define these — be ready with a concrete example of each and understand why each is useful. "Because it's a pillar of OOP" is not an answer.

1.5 Benefits and Drawbacks of OOP

Benefits

- **Modularity:** Each class is a self-contained unit. Understanding one doesn't require understanding everything.
- **Reusability:** Well-designed classes can be used across many projects; inheritance and composition let you reuse behavior.
- **Maintainability:** Encapsulation limits the blast radius of changes.
- **Natural modeling:** Real-world domains map well to objects.
- **Team-friendly:** Different developers can work on different classes with minimal collisions.

Drawbacks (Yes, These Exist)

- **Overhead:** Objects use more memory than plain structs. Method dispatch has an indirection cost. In performance-critical code (games, embedded), sometimes procedural or data-oriented design wins.
- **Overengineering risk:** Deep inheritance hierarchies and factory-of-factory-of-factory patterns make simple problems complicated.
- **Not always the best fit:** Data processing pipelines, math-heavy code, and stateless services often fit functional or procedural styles better.
- **Steep initial learning curve:** Understanding inheritance, polymorphism, and patterns takes time.

Chapter 2

Classes and Objects

This is where OOP starts to become concrete. Let's dive into the anatomy of a class, how objects come into existence, and the mechanics of state and behavior.

2.1 Anatomy of a Class

A class definition has several parts. In UML notation, a class is drawn as a rectangle divided into three compartments: name at the top, attributes in the middle, operations (methods) at the bottom.

UML Class Notation

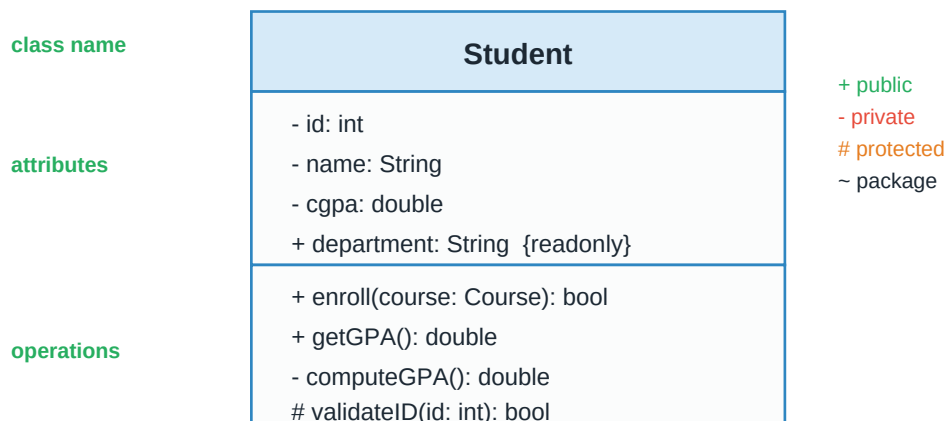


Figure: Standard UML class notation — you'll see this everywhere

The Compartments

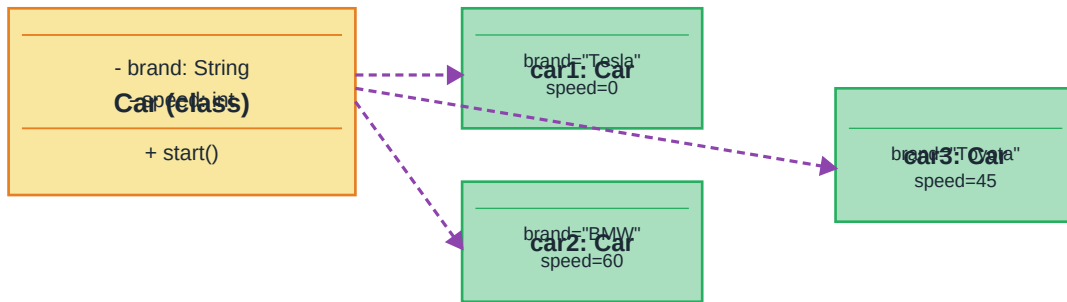
- **Class name:** By convention, singular and PascalCase — Student, BankAccount, ShoppingCart. Not Students, not shopping_cart.
- **Attributes:** The data fields. Format: `visibility name: type`. The visibility markers: + public, - private, # protected, ~ package.
- **Operations:** The methods. Format: `visibility name(parameters): returnType`.

Modifiers can appear as annotations in braces: {readonly}, {static}, {abstract}. Static members are usually underlined in UML.

2.2 From Class to Object — Instantiation

Class = Blueprint | Object = Instance

«instantiate»



One class → many objects. Same structure, different state.

In code: `Car car1 = new Car();` | `car1 = Car()` in Python

Each object has its OWN copy of instance attributes

Figure: A class is a blueprint; objects are instances built from it

Creating an object from a class is called **instantiation**. Different languages have slightly different syntax:

```

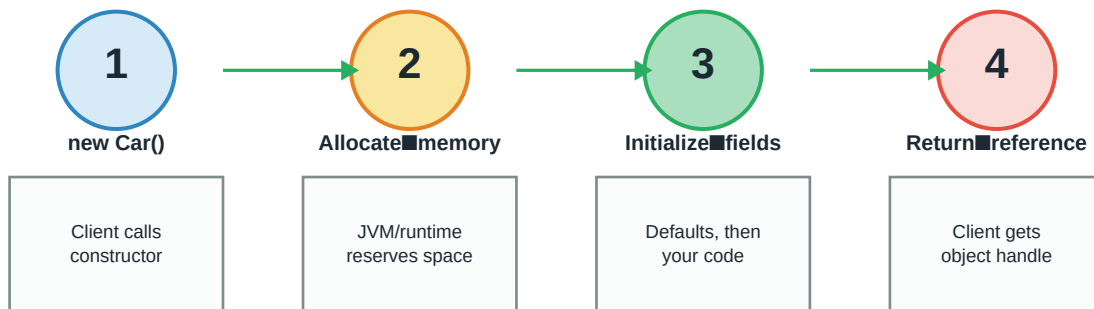
Java, C#, C++: Car car = new Car();
Python: car = Car()
JavaScript: const car = new Car();
Kotlin: val car = Car()
Ruby: car = Car.new
  
```

Each instantiation produces a new object with its own state. Modifying car1's speed doesn't affect car2's speed — they're independent.

2.3 Constructors

A **constructor** is a special method that runs when an object is created. Its job: initialize the object's state so it's ready to use.

Constructor: Called When Object is Created



Three constructor flavors: default (no args), parameterized (custom), copy

If you don't define one, the compiler adds a public no-arg constructor

Once you define any constructor, no default is added — a common source of bugs

Figure: Object creation — four steps under the hood

Types of Constructors

- **Default (no-arg) constructor:** Takes no parameters. Every field gets its default value (0 for int, null for objects, etc.).
- **Parameterized constructor:** Takes arguments to initialize fields with specific values.
- **Copy constructor:** Takes an existing object and creates a copy of it. In Java: `Point p2 = new Point(p1);`. In C++, this is a fundamental feature.

```

class Person {
    String name;
    int age;

    public Person() { // default
        this.name = 'Unknown';
        this.age = 0;
    }

    public Person(String n, int a) { // parameterized
        this.name = n;
        this.age = a;
    }

    public Person(Person other) { // copy
        this.name = other.name;
        this.age = other.age;
    }
}
  
```

Important: Common bug: if you define ANY constructor, most languages stop providing the default no-arg one for you. If your code (or a framework) relies on `new Person()`, it'll break the moment you add a parameterized constructor. Explicitly define both.

Constructor Chaining

One constructor can call another — either in the same class using `this(...)`, or in the parent class using `super(...)`. Reduces duplication.

```
public Person(String name) {
    this(name, 0); // chain to the 2-arg constructor
}
public Person(String name, int age) {
    this.name = name;
    this.age = age;
}
```

2.4 Destructors and Object Lifecycle

The opposite of a constructor. A **destructor** (or finalizer) runs when an object is being destroyed. Different languages handle this VERY differently — a common interview topic.

- **C++:** Explicit destructors. Called automatically when an object goes out of scope or is explicitly deleted. Deterministic — you know exactly when it runs. Foundation of **RAII** (Resource Acquisition Is Initialization) — a huge C++ idiom.
- **Java:** Has `finalize()`, but it's deprecated. GC might call it before an object is collected — or not at all. Use `try-with-resources` or `AutoCloseable` instead.
- **Python:** Has `__del__`, called when reference count drops to zero. Also unreliable in the face of reference cycles. Use context managers (`with`) instead.
- **C#:** Has finalizers via `~ClassName()`, but prefer `IDisposable` and `using` blocks.

Key Insight: Modern advice across languages: don't rely on destructors to release critical resources (file handles, network connections, DB connections). Use explicit RAII/dispose patterns.

2.5 The this / self Reference

Inside a method, how does the object refer to itself? Through a special variable — usually called `this` (Java, C++, C#, JavaScript) or `self` (Python, Ruby).

```
class Circle {
    double radius;
    public Circle(double radius) {
        this.radius = radius; // disambiguates field from parameter
    }
    public double area() {
        return 3.14159 * this.radius * this.radius;
    }
}
```

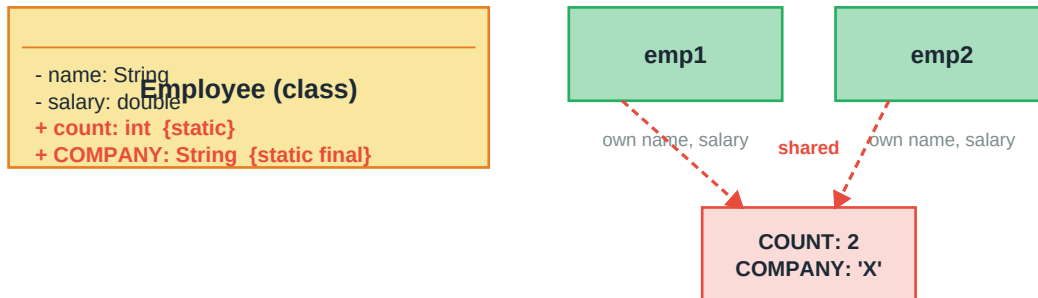
Uses of `this`:

- Disambiguate a field from a parameter with the same name.
- Pass the current object to another method or constructor.
- Return the current object for method chaining (fluent APIs).
- Chain to another constructor via `this(...)`.

2.6 Static Members

Normal (instance) members live in the object — each object has its own copy. **Static** members live in the CLASS itself — one shared copy for everyone.

Static vs Instance Members



Instance: one copy per object (name, salary)

Static: one copy shared by all objects & the class itself

Access via `ClassName.staticMember`, not `object.member`

Use for: constants, counters, utility methods, factory methods

Figure: Instance members are per-object; static members are shared class-wide

Static Fields

Common uses: counters (how many objects created so far?), configuration constants (`Math.PI`), singletons (Chapter 8), and caching.

Important: Static state is essentially global. It's thread-safety-hostile, hard to test (can't reset between tests), and often a symptom of poor design. Use sparingly.

Static Methods

Methods that don't need an instance to run. They can't access instance fields or non-static methods of the class (since there's no `this`). Common uses:

- Utility functions: `Math.sqrt(x)`, `Integer.parseInt(s)`
- Factory methods (Chapter 8).
- Entry points: `public static void main(String[] args)`.

Chapter 3

Encapsulation

The first of the four pillars. Encapsulation is arguably the most important — get it wrong and every other design decision suffers. Let's understand what it really means beyond "data hiding".

3.1 What is Encapsulation?

Encapsulation is the bundling of data (fields) and the methods that operate on that data into a single unit (the class), combined with restricting direct access to some of the object's internals. The class controls what the outside world can and cannot do.

Encapsulation: Data Hidden Behind a Public Interface

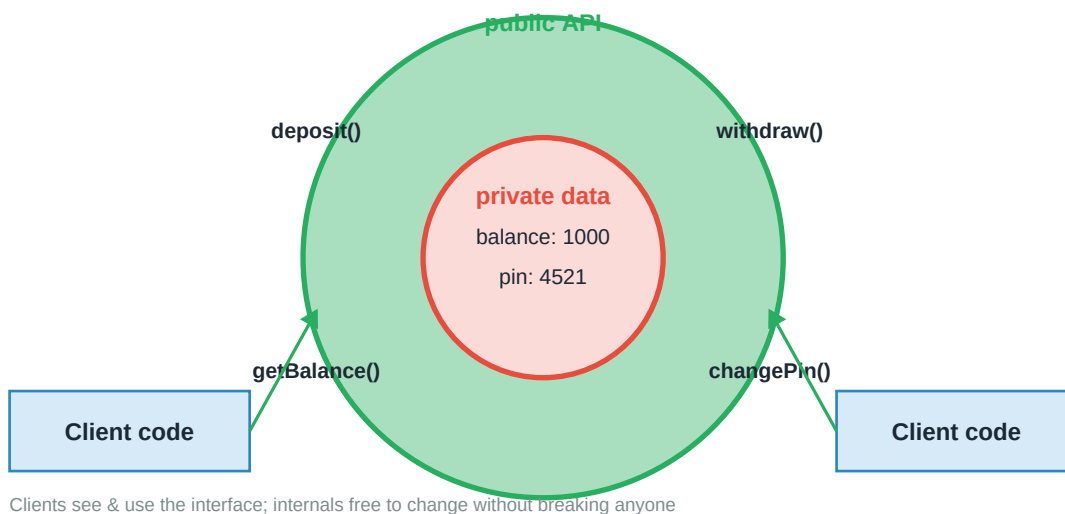


Figure: Encapsulation — private data protected by a public API

Two closely related ideas often conflated:

- **Data hiding:** Making fields private so clients can't read/write them directly.
- **Information hiding:** The broader idea (Parnas, 1972) that a module should hide its IMPLEMENTATION DECISIONS behind a stable interface. Encapsulation is one mechanism to achieve it.

Analogy: Think of a car. When you drive, you see a steering wheel, gas pedal, and gauges. You don't reach into the engine to adjust fuel injection timing. That's encapsulation. The car's designers can completely redesign the engine — as long as the pedal-and-wheel interface stays the same, you're unaffected.

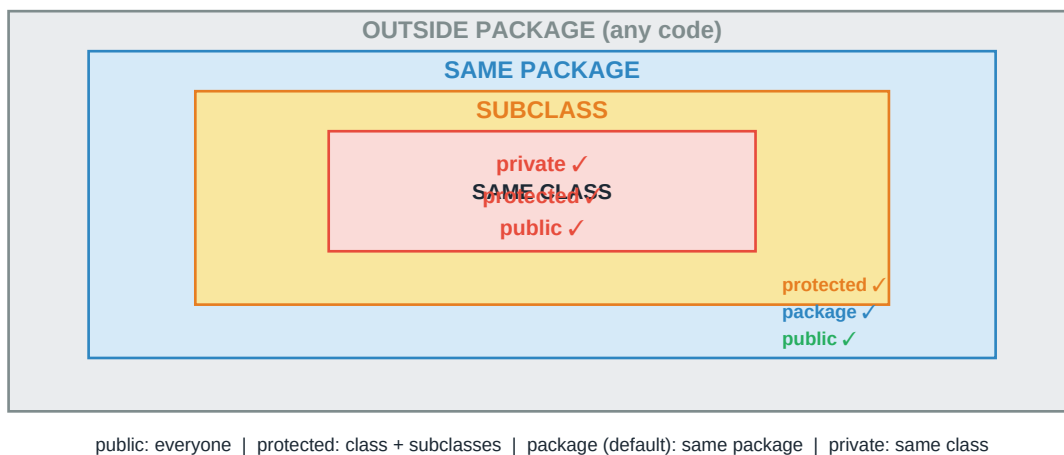
3.2 Why Encapsulate?

- **Invariants:** The class can enforce rules. A BankAccount can guarantee balance is never negative by validating in `withdraw()` — if balance were public, anyone could set it to -1000.
- **Change safely:** Change internal representation without breaking clients. If `firstName` and `lastName` are private and accessed via `getFullName()`, we can later store them as one string without breaking anyone.
- **Reduce coupling:** External code depends only on the public interface, not implementation details.
- **Reason locally:** To understand a class, read the class. External behavior can't depend on internal fields.

3.3 Access Modifiers

The mechanisms that enforce encapsulation vary by language, but the general categories are similar.

Access Modifiers: Who Can See What?



Rule of thumb: start private, widen only when necessary

Figure: Access modifier scopes — from strictest to loosest

- **private:** Access only from inside the same class. Strongest protection. Default for state.
- **protected:** Access from the class and its subclasses. Slightly wider — for things subclasses genuinely need.
- **package-private (Java default, no keyword):** Access from same package. Useful when a set of cooperating classes need to see each other's internals.
- **public:** Access from anywhere. The interface. Keep this small.

Language-Specific Notes

- **Java:** Four levels — private, package (default), protected, public. Strictly enforced.
- **C++:** Three levels — private, protected, public. Enforced. Also has `friend` declarations to grant selective access.
- **C#:** Six levels — private, protected, internal, protected internal, private protected, public.
- **Python:** No enforcement — only conventions. `__name` means "internal, don't touch", `__name` triggers name mangling. "We're all adults here" philosophy.

Key Insight: Rule of thumb: start every field as private. Widen only when a legitimate need arises. It's much easier to open up access later than to lock it down (which breaks callers).

3.4 Getters and Setters

When a field must be private but clients need to read/write it, you provide **accessor methods**: getters (read) and setters (write). Together they replace direct field access.

```
public class BankAccount {
    private double balance; // hidden

    public double getBalance() { // read via getter
        return balance;
    }

    public void setBalance(double newBalance) { // write via setter
        if (newBalance < 0) throw new IllegalArgumentException();
        this.balance = newBalance;
    }
}
```

The setter can validate input, log changes, notify listeners, or reject bad values. The getter can return a computed value or a defensive copy of a mutable field. All of this is impossible with a public field.

Getters/Setters Aren't Sacred

Important: Some code has a getter and setter for every private field. This isn't encapsulation — it's a public field with extra typing. Real encapsulation is about EXPOSING BEHAVIOR, not exposing data.

Bad: `account.setBalance(account.getBalance() - 100);` — client is doing the arithmetic.

Better: `account.withdraw(100);` — the operation lives in the class where it belongs. The class can enforce that withdrawals don't exceed the balance, deduct fees, log the transaction, etc.

Key Insight: Think in terms of "what should this object let clients DO" — not "what data does this object hold that I need to expose".

3.5 Immutability

An **immutable** object is one whose state cannot change after construction. All fields are set in the constructor and never modified. There are no setters. Instead of mutating, you create new objects.

```
public final class Point {
    private final int x;
    private final int y;
    public Point(int x, int y) {
        this.x = x; this.y = y;
    }
    public int getX() { return x; }
    public int getY() { return y; }
    public Point translate(int dx, int dy) {
        return new Point(x + dx, y + dy); // NEW object
    }
}
```

```
}
```

Benefits of Immutability

- **Thread-safe by default:** No mutations = no data races.
- **Safe to share:** Pass immutable objects around without defensive copies.
- **Safe as map keys:** Their hash never changes; they can't sabotage the map.
- **Easier to reason about:** No hidden state changes.
- **Cache-friendly:** Value equality can be based on content; identical objects can be reused.

Java examples of immutable classes: `String`, `Integer`, all boxed primitives, `LocalDate`, `LocalDateTime`.

Key Insight: Effective Java, Item 17: Minimize mutability. Prefer immutable objects unless you have concrete reasons for mutability.

Chapter 4

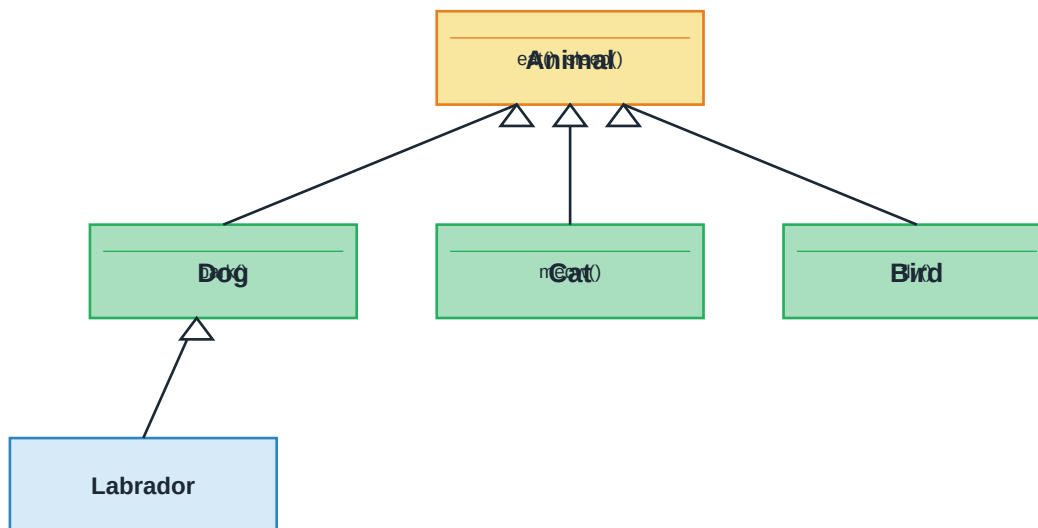
Inheritance

Inheritance lets one class reuse and extend another. It's OOP's most famous — and most abused — feature. This chapter covers how it works, its pitfalls, and when to prefer composition.

4.1 What is Inheritance?

Inheritance is a mechanism where a new class (subclass, derived class, child) inherits attributes and behaviors from an existing class (superclass, base class, parent). The subclass gets everything the parent has and can add more or override existing behavior.

Inheritance Hierarchy (IS-A Relationship)



Subclasses inherit parent's non-private members AND can add/override their own

Figure: Inheritance hierarchy — a tree of specialization

The relationship is called **IS-A**: a Dog IS-A Animal. Anywhere an Animal is expected, a Dog can be provided. This is the essence of subtyping.

```
// Java syntax
class Animal {
    String name;
    void eat() { ... }
}

class Dog extends Animal { // Dog inherits from Animal
    void bark() { ... } // Dog adds its own method
}

Dog d = new Dog();
```



```
d.eat(); // inherited from Animal
d.bark(); // Dog's own
```

4.2 Types of Inheritance

- **Single:** One parent, one child. Java, C#, Kotlin allow only this for classes.
- **Multilevel:** Grandparent → parent → child. All languages support this.
- **Hierarchical:** One parent, multiple children. Common — Animal → Dog, Cat, Bird.
- **Multiple:** One child with multiple parents. Supported by C++, Python, but not Java/C# for classes (only interfaces). Causes the diamond problem.
- **Hybrid:** Combination of multiple and hierarchical.

4.3 What Gets Inherited?

- **Public and protected fields and methods** are inherited.
- **Private members** exist in the parent but are not directly accessible in the child.
- **Constructors are NOT inherited.** Every class defines its own.
- **Static members** are accessible from the child via the class name.

Constructor Chaining

When you construct a subclass, the parent's constructor runs **FIRST**. This is implicit — `super ( )` is called for you if you don't call it explicitly.

```
class Dog extends Animal {
    private String breed;
    public Dog(String name, String breed) {
        super(name); // must be first line if used
        this.breed = breed;
    }
}
```

Important: If the parent has no no-arg constructor and you don't explicitly call `super ( . . . )` with the right args, you get a compile error. This trips people up when refactoring parent classes.

4.4 Method Overriding

Subclasses can provide a new implementation for an inherited method. This is called **overriding**. The subclass's version replaces the parent's for instances of the subclass.

```
class Animal {
    public String sound() { return 'Some sound'; }
}
class Dog extends Animal {
    @Override
```

```
public String sound() { return 'Woof'; }
}

Animal a = new Dog();
a.sound(); // 'Woof' – Dog's version, chosen at runtime
```

Rules for a valid override:

- Same method name and parameter list.
- Return type same or a subtype (covariant returns).
- Access modifier same or less restrictive.
- Throws declaration cannot add new checked exceptions.

Key Insight: Always use `@Override` annotation (Java) or `override` keyword (C#). The compiler will catch mistakes like typos in method names that would otherwise silently create a new method instead of overriding.

4.5 Composition vs Inheritance — The Big Debate

Inheritance (IS-A) vs Composition (HAS-A)

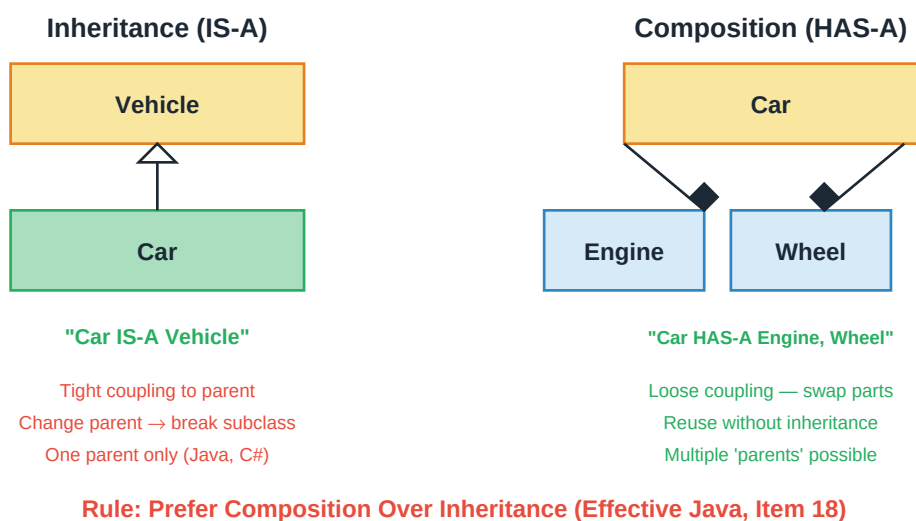


Figure: Inheritance (IS-A) vs Composition (HAS-A)

Modern OOP wisdom: **prefer composition over inheritance**. Why? Inheritance has hidden costs that make code fragile.

The Fragile Base Class Problem

A subclass depends on the **SPECIFIC IMPLEMENTATION** of its parent, not just its interface. If the parent changes internal behavior — even in a backward-compatible way — subclasses can break in surprising ways.

Example: Suppose you extend Java's `ArrayList` to count how many times `add()` is called. You override `add()` and `addAll()` to increment a counter. But `ArrayList`'s `addAll()` internally calls `add()` for each element. Now every `addAll()` double-counts. The parent's internal implementation leaked into your subclass.

When to Use Which

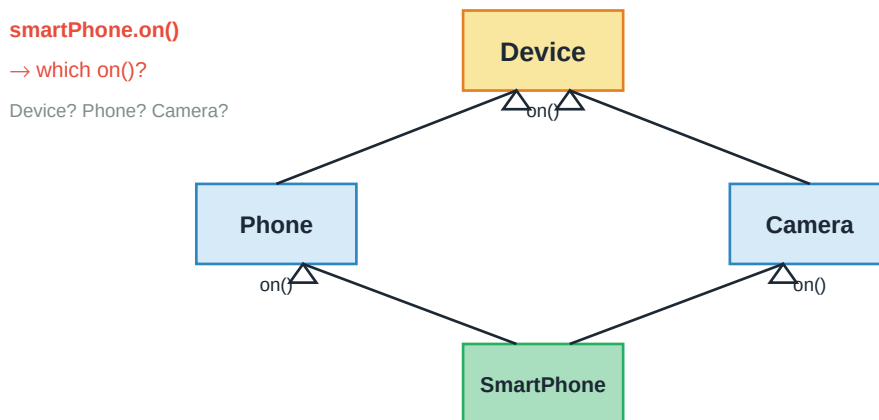
- **Inheritance:** When there's a clear IS-A relationship AND the parent was designed for extension (documented, protected extension points, stable).
- **Composition:** When there's a HAS-A relationship, or when you just want to reuse behavior without a strong IS-A.

"Prefer composition" doesn't mean "never inherit" — it means default to composition, use inheritance when it genuinely fits.

4.6 Multiple Inheritance and the Diamond Problem

Some languages (C++, Python) let a class inherit from multiple parents. This is powerful — a SmartPhone can inherit from both Phone and Camera. But it creates a nasty ambiguity called the **diamond problem**.

The Diamond Problem in Multiple Inheritance



Java/C# solve this by disallowing multiple class inheritance (use interfaces).

Figure: Diamond problem — which parent's method is called?

How Languages Solve It

- **Java, C#, Kotlin:** Forbid multiple class inheritance. A class can implement multiple interfaces, but interfaces don't carry state (in Java 8+, default methods have rules to prevent ambiguity).
- **C++:** Allows multiple inheritance. Ambiguities must be resolved explicitly with scope resolution (Phone : : on ()). Provides **virtual inheritance** to share a common base.
- **Python:** Uses **MRO (Method Resolution Order)** — C3 linearization algorithm. SmartPhone.__mro__ gives you the exact order Python will search.

4.7 Preventing Inheritance

Sometimes you don't want your class to be extended — or a specific method to be overridden. Different languages provide keywords for this:

- **Java:** `final` — `final` class can't be extended; `final` method can't be overridden.
- **C#:** `sealed` — same idea.
- **Kotlin:** Classes are `final` BY DEFAULT — you must explicitly mark open to allow extension. This is a strong stance.

Key Insight: Design for inheritance AND document it — or prohibit it. Halfway measures create fragile base classes. String and Integer in Java are final for good reason.

Chapter 5

Polymorphism

The third pillar. "Polymorphism" means "many forms" — one interface, many behaviors. It's what makes OOP truly powerful, and it's asked in nearly every OOP interview.

5.1 What is Polymorphism?

Polymorphism is the ability of an entity (variable, method, object) to take on many forms. In OOP, it usually means: the same method call can invoke different behavior depending on the actual type of the object at runtime.

Analogy: Imagine a Universal TV remote. You press "Volume Up" and different TVs react — Sony, Samsung, LG — each does its own thing internally, but the remote's interface is the same. The remote is polymorphic: one action, many actual behaviors depending on what's on the receiving end.

Types of Polymorphism

- **Compile-time (static) polymorphism:** Decided by the compiler based on argument types. Achieved via method overloading and operator overloading.
- **Runtime (dynamic) polymorphism:** Decided at runtime based on the actual object type. Achieved via method overriding — the real magic of OOP.
- **Parametric polymorphism:** Same code works with different types via generics/templates. `List<Integer>` and `List<String>` share the same `List` code.
- **Ad-hoc polymorphism:** Overloading. The behavior for each type is defined separately.

5.2 Method Overloading (Compile-Time)

Overloading is having multiple methods in the same class with the same name but different parameter lists. The compiler picks which one to call based on the arguments provided.

Compile-Time Polymorphism (Overloading)

Same method name, different parameter lists

<code>int add(int a, int b)</code>	returns a + b (ints)
<code>double add(double a, double b)</code>	returns a + b (doubles)
<code>int add(int a, int b, int c)</code>	returns a + b + c
<code>String add(String a, String b)</code>	returns concatenation

Compiler picks the right one based on argument types → decided at COMPILE TIME

Figure: Overloading — multiple methods, same name, different signatures

Rules for valid overloading:

- Same method name.
- Different parameter list (number, types, or order).
- Return type alone is NOT enough to distinguish overloaded methods.

Important: Common trick question: Can you overload a method by changing only its return type? **Answer: No.** The compiler needs to pick the method based on the call site, and the return type isn't visible at the call site. Only the parameter list distinguishes overloaded methods.

Java's `System.out.println()` is a great example — overloaded for `int`, `double`, `String`, `Object`, and many more. Same call, right method chosen by argument type.

5.3 Method Overriding (Runtime)

Overriding happens across classes in an inheritance chain. A subclass provides its own implementation of a method already defined in its parent. The version that runs depends on the ACTUAL OBJECT type, not the reference type.

Runtime Polymorphism (Overriding)

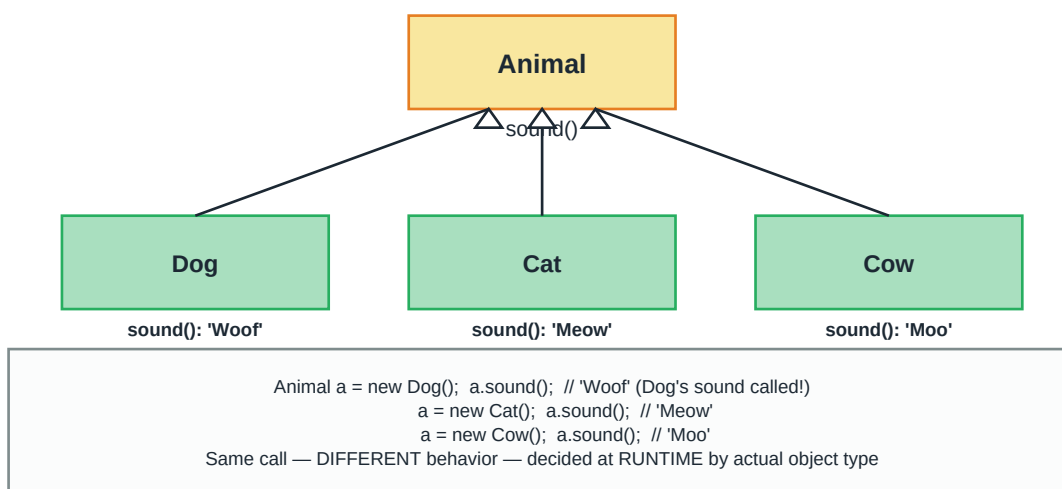


Figure: Overriding — actual object type decides which method runs

This is called **dynamic method dispatch** or **late binding**. The decision of which method to call is deferred until runtime.

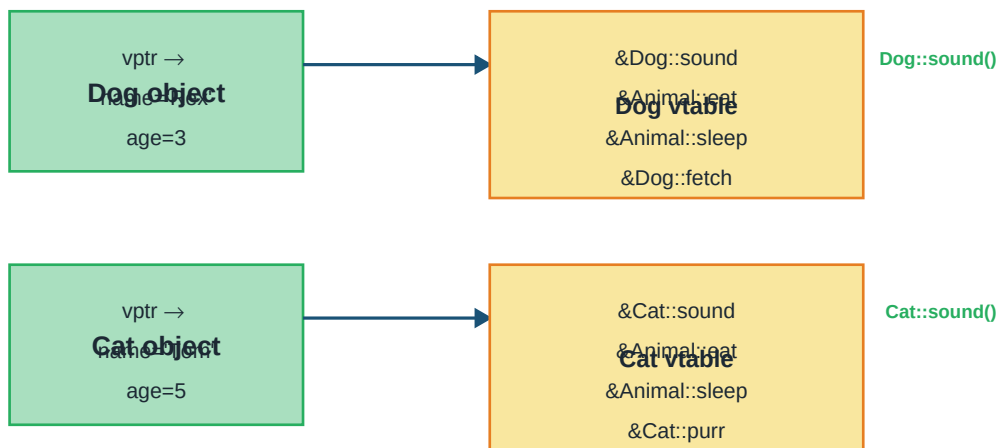
Overloading vs Overriding — The Killer Comparison

Aspect	Overloading	Overriding
Where	Same class	Parent → child (across classes)
Parameters	Must differ	Must match exactly
Return type	Can differ	Same or covariant
Access modifier	Any	Same or wider
Decision	Compile-time (static)	Runtime (dynamic)
Static methods?	Can be overloaded	Cannot be overridden (only hidden)
Polymorphism type	Compile-time	Runtime

5.4 Dynamic Dispatch and vtables

How does the runtime know which overridden method to call? Through a mechanism called the **virtual function table (vtable)**. This is a foundational C++/Java/C# concept.

Virtual Function Table (vtable) — How Dispatch Works



Each object has **vptr** to its class's **vtable**. Method call = **vptr lookup** + call.

One indirection = the price of polymorphism in C++/Java/C#

Figure: Every object carries a vptr to its class's vtable

Each class has a vtable — an array of function pointers, one per virtual (overridable) method. Each object has a hidden **vptr** (virtual pointer) that points to its class's vtable. When you call a virtual method:

- Follow the object's vptr → get the vtable.
- Look up the method by its slot in the vtable.
- Call the function at that slot.

This adds ONE indirection compared to a direct call — usually negligible, but relevant in extremely hot code paths. It's why calling virtual methods is slightly slower than calling non-virtual ones.

Language Defaults

- **Java, C#, Kotlin:** All methods are virtual by default. Use `final` (Java) or `sealed` (C#) to make them non-virtual.
- **C++:** Methods are non-virtual by default. Add `virtual` keyword to make them overridable.
- **Python:** All methods are effectively virtual (attribute lookup happens at runtime).

Important: In C++, forgetting `virtual` is a classic bug. If a base method isn't virtual, calling it through a base pointer runs the BASE version even if the object is a derived type. No polymorphism. Same for destructors — a non-virtual base destructor causes memory leaks.

5.5 Upcasting and Downcasting

Since a Dog IS-A Animal, we can treat a Dog object as if it were an Animal. This is **upcasting** — moving up the hierarchy toward more general types. It's always safe.

```

Dog d = new Dog();
Animal a = d; // upcast, implicit and safe
a.eat(); // Animal methods available
// a.bark(); // NO – Animal doesn't know about bark
  
```


Downcasting — moving down the hierarchy — is the opposite. It's NOT automatically safe: not every `Animal` is a `Dog`. You must explicitly cast, and the runtime checks.

```
Animal a = new Dog();
Dog d = (Dog) a; // downcast, explicit
d.bark(); // works, because a really IS a Dog

Animal a2 = new Cat();
Dog d2 = (Dog) a2; // ClassCastException at runtime
```

To safely check before casting, use `instanceof` in Java, `is` in C# and Kotlin:

```
if (a instanceof Dog) {
    Dog d = (Dog) a;
    d.bark();
}
```

Key Insight: Excessive downcasting ("if animal instanceof Dog... else if instanceof Cat...") is a code smell — usually a sign that polymorphism isn't being used properly. Consider adding a method to `Animal` that each subclass overrides, and let dispatch handle the dispatch.

Chapter 6

Abstraction — Abstract Classes and Interfaces

The fourth pillar. Abstraction is about hiding complexity by exposing only what's essential. In OOP, we achieve this through abstract classes and interfaces — the two most confused topics in OOP interviews.

6.1 What is Abstraction?

Abstraction means representing essential features without including background details. When you drive a car, you use accelerator, brake, and steering — a simplified interface to a complex machine. You don't need to think about the engine's combustion cycles or the ABS module's timing.

In OOP, abstraction shows up in two related ways:

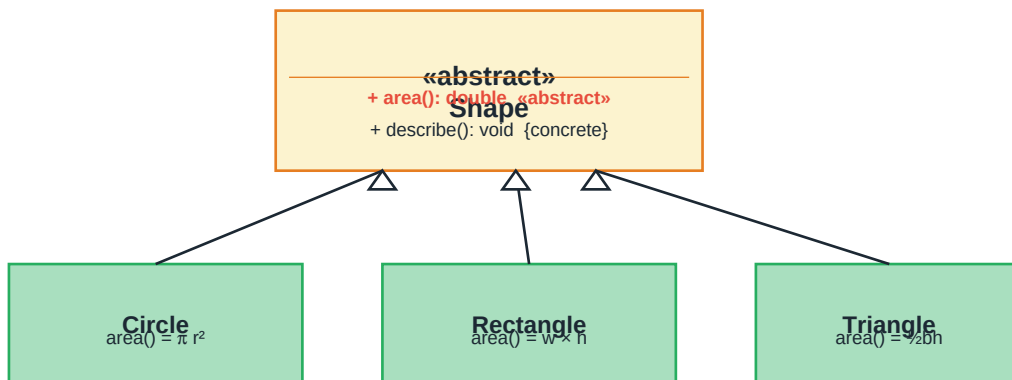
- **Data abstraction:** Hide internal representation. Clients use the object without knowing HOW it stores its data. This is where encapsulation and abstraction overlap.
- **Behavioral abstraction:** Define WHAT a class does without saying HOW. Achieved via abstract classes and interfaces.

Key Insight: Encapsulation vs Abstraction: Encapsulation is the MECHANISM (hiding fields, exposing methods). Abstraction is the DESIGN GOAL (simplify the model, hide implementation). Encapsulation helps you achieve abstraction.

6.2 Abstract Classes

An **abstract class** is a class that cannot be instantiated directly — only extended. It usually contains at least one **abstract method** (declared without an implementation), forcing subclasses to provide the concrete implementation.

Abstract Class — Partial Implementation



Abstract methods **MUST** be overridden by concrete subclasses.

You can't instantiate **Shape** directly — `new Shape()` is illegal.

Figure: Abstract Shape — subclasses must implement `area()`

```

abstract class Shape {
    private String color;

    public Shape(String color) { // constructor OK
        this.color = color;
    }

    public abstract double area(); // ABSTRACT – no body

    public void describe() { // concrete method
        System.out.println(color + ' with area ' + area());
    }
}

class Circle extends Shape {
    private double radius;
    public Circle(String color, double r) {
        super(color); this.radius = r;
    }
    @Override
    public double area() { return Math.PI * radius * radius; }
}
  
```

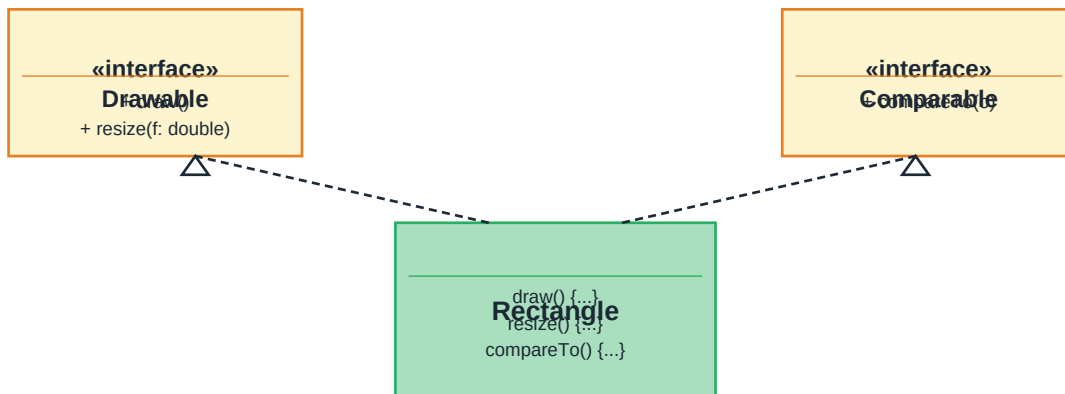
Key Points About Abstract Classes

- Can have BOTH abstract and concrete methods.
- Can have constructors (called by subclasses via `super()`).
- Can have fields (state) — including private state.
- Cannot be instantiated: `new Shape()` is illegal.
- A concrete subclass must implement ALL inherited abstract methods.
- In Java, abstract classes cannot be `final` (that would be nonsensical).

6.3 Interfaces

An **interface** is a contract — a pure specification of what methods a class must provide, with no implementation. It says WHAT a class does, not HOW.

Interfaces — Pure Contract, No Implementation



A class can implement MULTIPLE interfaces — Java's answer to multiple inheritance

Interfaces express capabilities: 'this thing can be drawn', 'this thing can be compared'

Since Java 8: interfaces can have 'default' methods with implementation

Figure: A class can implement multiple interfaces — Java's answer to multiple inheritance

```

interface Drawable {
    void draw();
    void resize(double factor);
}

interface Comparable<T> {
    int compareTo(T other);
}

class Rectangle implements Drawable, Comparable<Rectangle> {
    @Override public void draw() { ... }
    @Override public void resize(double f) { ... }
    @Override public int compareTo(Rectangle other) { ... }
}
  
```

Key Points About Interfaces (Classic)

- All methods are implicitly public and abstract.
- All fields are implicitly public static final (constants).
- No constructors — you can't instantiate an interface.
- A class can implement MULTIPLE interfaces — powerful.

Modern Interfaces (Java 8+)

Java 8 gave interfaces two new capabilities:

- **Default methods:** Methods with a default implementation. Allows adding methods to an interface without breaking classes that already implement it.

- **Static methods:** Utility methods that belong to the interface (like `Comparator.naturalOrder()`).
- **Private methods (Java 9+):** For sharing code between default methods without exposing it.

```
interface Vehicle {
    void start();

    default void honk() { // default method
        System.out.println('Beep!');
    }

    static Vehicle bicycle() { // static factory
        return new Bicycle();
    }
}
```

6.4 Abstract Class vs Interface — The Interview Classic

Abstract Class vs Interface

	Abstract Class	Interface
Purpose	Partial impl., IS-A	Pure contract, CAN-DO
Methods	Abstract + concrete	Abstract (+default in Java 8+)
Fields	Any (inc. state)	public static final only
Constructors	Yes	No
Multiple inheritance	No (one parent)	Yes (many interfaces)
Instantiation	Cannot (with new)	Cannot (with new)
Use when	Common code + variation	Multiple unrelated types

Figure: Abstract Class vs Interface — full comparison

When to Use Each

Use an abstract class when:

- Multiple related classes share significant common CODE (not just contract).
- You need to declare non-public members, state, or constructors.
- You want to provide a template with hook methods for subclasses to fill in (Template Method pattern).

Use an interface when:

- You want to define a capability that unrelated classes can implement (Comparable, Serializable).
- A class needs to conform to multiple contracts (multiple inheritance).
- You want the loosest possible coupling.

Key Insight: Modern rule of thumb: Default to interfaces. Reach for abstract classes only when you need to share genuinely reusable code or state. Java's own APIs increasingly favor interfaces with default methods over abstract classes.

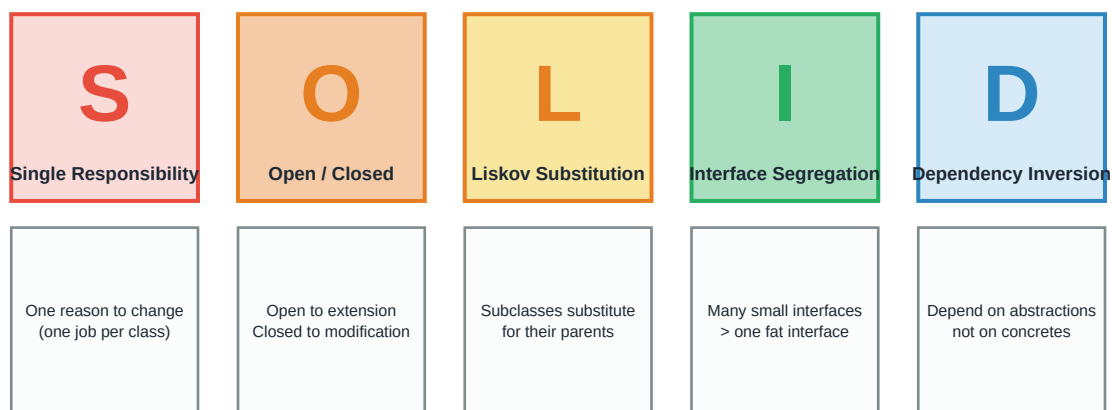
Chapter 7

The SOLID Principles

Everyone can write code that a computer understands. SOLID principles help you write code that OTHER HUMANS can understand, maintain, and extend. These five principles are the interview classic — and unlike some interview trivia, they're actually useful in daily work.

7.1 Overview

The SOLID Principles of OOD



Robert C. Martin's five principles for maintainable OO software

Following SOLID gives you code that resists rot and welcomes change

Figure: Robert C. Martin's five principles for maintainable object-oriented design

The SOLID acronym was coined by Michael Feathers, from principles articulated by Robert C. Martin ("Uncle Bob"). Together they form a coherent philosophy of OO design that prioritizes maintainability, extensibility, and testability.

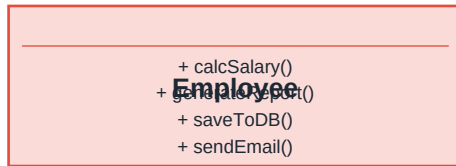
Understand each principle DEEPLY. In interviews, you'll be expected to give an example for each — not just recite the acronym.

7.2 S — Single Responsibility Principle

"A class should have one, and only one, reason to change."

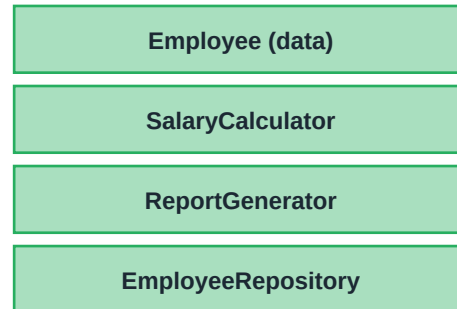
S — Single Responsibility Principle

■ Violates SRP



3 reasons to change:
reports, DB, email

✓ Follows SRP



Each class = one job

'A class should have only one reason to change'

Not: 'one method' — but ONE COHESIVE RESPONSIBILITY

Figure: SRP violation and fix — split by responsibility

Every class should have a single, well-defined responsibility. Not "only one method" — "one thing to worry about." The test: how many different STAKEHOLDERS might request changes to this class?

Example: An Employee class that (1) computes salary, (2) generates reports, and (3) saves to database has three responsibilities. Accounting might request salary changes; management might request report changes; the DBA might request database schema changes. Three stakeholders → three reasons to change → three classes.

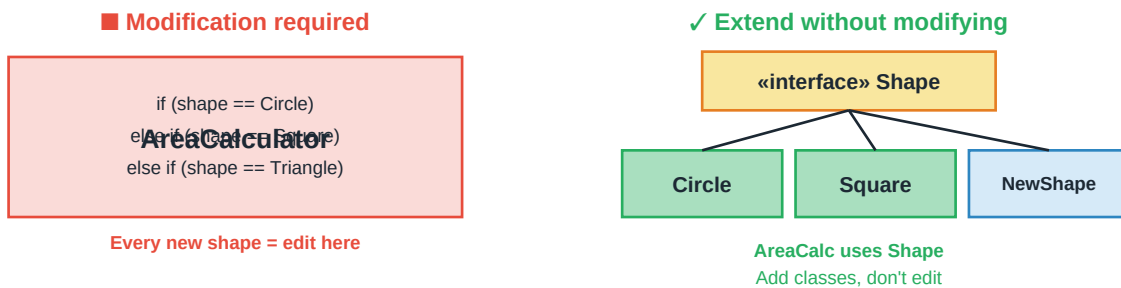
Why It Matters

- Classes stay small and focused — easier to read.
- Changes for one reason don't risk breaking code for another reason.
- Testing becomes simpler — smaller units, fewer test cases.
- Reuse becomes possible — a focused class is easier to reuse elsewhere.

7.3 O — Open/Closed Principle

"Software entities should be OPEN for extension but CLOSED for modification."

O — Open / Closed Principle



'Software entities should be OPEN for extension, CLOSED for modification'

New behavior via new subclasses/implementations, not by editing existing code

Why? Editing existing tested code risks regressions. Adding new code doesn't.

Figure: OCP — extend behavior via new classes, not by editing existing ones

You should be able to add new functionality without changing existing, tested code. The mechanism: abstraction (interfaces, abstract classes) and polymorphism.

Example: Classic anti-pattern: a giant if-else chain that grows every time a new type is added. `if (shape == CIRCLE) ... else if (shape == SQUARE) ...`. Every new shape means editing this method — and risking regressions in the existing code.

Fix: create a Shape interface with an `area()` method. Each shape implements it. To add a new shape, add a new class. The area calculator is closed for modification, open for extension.

7.4 L — Liskov Substitution Principle

"Subtypes must be substitutable for their base types." Formulated by Barbara Liskov in 1987.

L — Liskov Substitution Principle

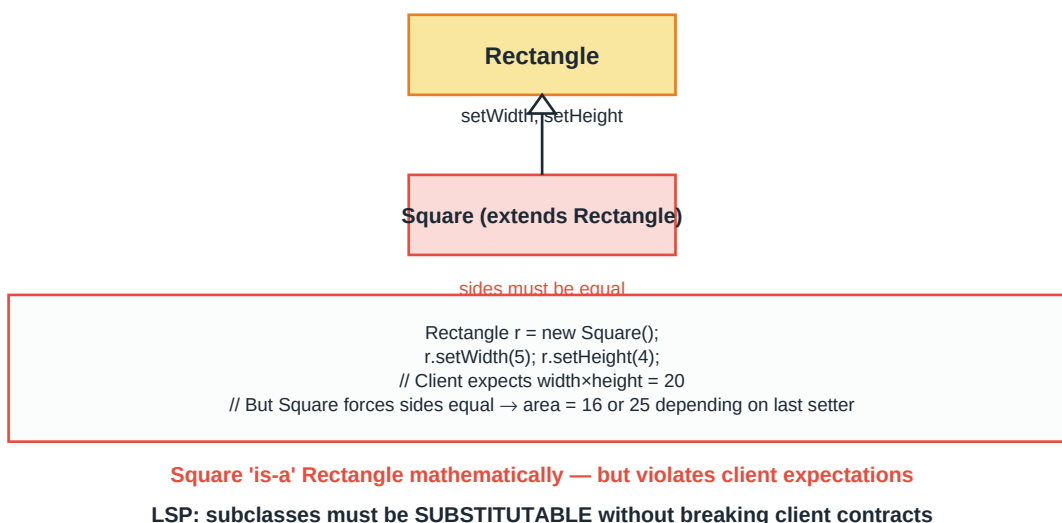


Figure: Classic LSP violation — Square breaks Rectangle's contract

If S is a subtype of T, then wherever the program uses T, we should be able to substitute S without breaking anything. This means the subclass must honor the parent's CONTRACT — not just its interface, but the promises the parent makes about behavior.

Example: The classic Rectangle-Square example. Mathematically, Square IS-A Rectangle. But Rectangle's contract usually says "width and height can be set independently." A Square that overrides setters to keep sides equal violates this. Suddenly `rect.setWidth(5); rect.setHeight(4); assert rect.area() == 20;` fails when `rect` is a Square. LSP violated.

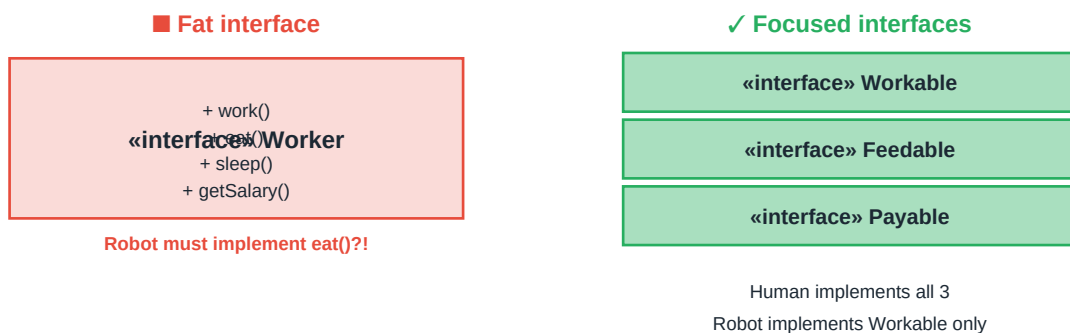
Concrete LSP Violations to Watch For

- Throwing new exceptions the parent doesn't throw.
- Strengthening preconditions (accepting fewer inputs).
- Weakening postconditions (guaranteeing less about outputs).
- Overriding a method to do nothing or throw `NotImplementedException`.

7.5 I — Interface Segregation Principle

"No client should be forced to depend on methods it does not use."

I — Interface Segregation Principle



'No client should be forced to depend on methods it does not use'

Split fat interfaces into role-based ones. Prefer many small over one huge.

Symptom to look for: 'throw new NotImplementedException()' or empty stub methods

Figure: ISP — split fat interfaces into role-based ones

A big fat interface with many methods forces every implementer to provide all of them, even the irrelevant ones. Better: many small, focused interfaces. Implement only what applies.

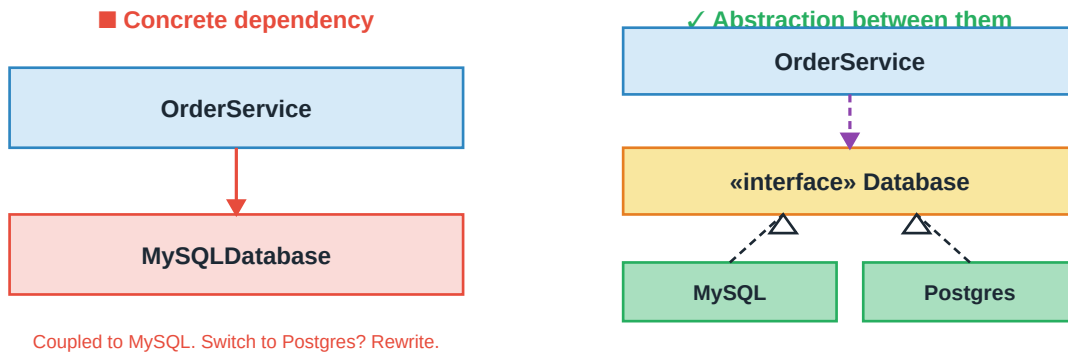
Example: A Worker interface with `work()`, `eat()`, `sleep()`, and `getSalary()`. A Human implements all four. But a Robot? A Robot works and doesn't need to eat or sleep or get paid. Making Robot implement empty `eat()` and `sleep()` methods (or throw `NotImplementedException`) is code smell. Split the interface into `Workable`, `Feedable`, `Restable`, `Payable` — implement what applies.

Key Insight: Interface pollution → dependency pollution. Even if a client only calls one method, they still depend on the whole interface. Split proactively.

7.6 D — Dependency Inversion Principle

"High-level modules should not depend on low-level modules. Both should depend on abstractions. Abstractions should not depend on details. Details should depend on abstractions."

D — Dependency Inversion Principle



'High-level modules should not depend on low-level; both should depend on ABSTRACTIONS'

Inject dependencies (constructor/setter) instead of creating them internally

This is the foundation of Dependency Injection frameworks (Spring, Guice, .NET DI)

Figure: DIP — an interface between high-level and low-level modules

Instead of a class DIRECTLY depending on another concrete class, both should depend on an **abstraction** (interface). This lets you swap implementations without changing the client.

Example: An OrderService that directly instantiates MySQLDatabase is tightly coupled to MySQL. Testing requires a real MySQL instance. Switching to PostgreSQL means editing OrderService.

Fix: OrderService depends on a Database interface. MySQLDatabase and PostgresDatabase both implement it. The choice of which one to use is made externally and injected into OrderService's constructor. For testing, inject a MockDatabase.

Dependency Injection (DI)

DIP is the principle. **Dependency Injection** is a common way to implement it: instead of a class creating its dependencies internally, they're passed in from outside — usually via constructor, setter, or a DI framework.

```
class OrderService {
    private final Database db;

    public OrderService(Database db) { // constructor injection
        this.db = db;
    }
}

// Client wires up:
Database db = new MySQLDatabase(); // or PostgresDatabase, or MockDatabase
OrderService service = new OrderService(db);
```

Key Insight: DI frameworks (Spring, Guice, .NET DI, Angular DI) automate this wiring based on config or annotations. Modern enterprise applications lean heavily on them.

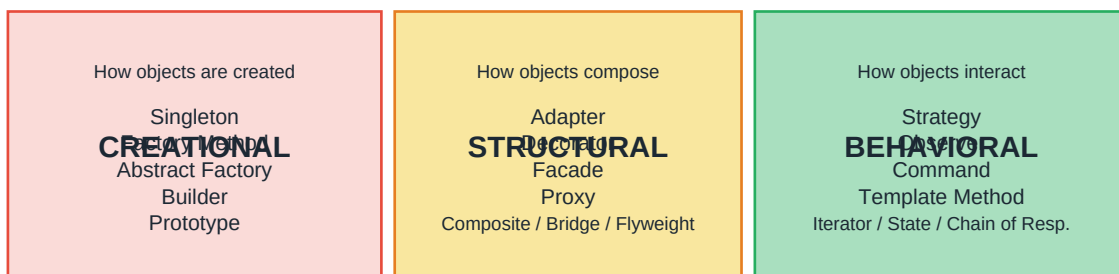
Chapter 8

Design Patterns

Design patterns are named, reusable solutions to common OOP design problems. They're not code — they're recipes you can apply. This chapter covers the most important ones from the Gang of Four (GoF) book, focused on what interviews actually ask.

8.1 What are Design Patterns?

Design Patterns — 23 GoF Patterns in 3 Categories



Named solutions to recurring OO design problems

From the 1994 'Gang of Four' book by Gamma, Helm, Johnson, Vlissides

In interviews: know a handful DEEPLY rather than all 23 shallowly

Figure: The 23 GoF patterns organized in three categories

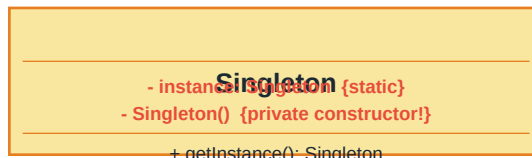
The **Gang of Four** (Erich Gamma, Richard Helm, Ralph Johnson, John Vlissides) published "Design Patterns: Elements of Reusable Object-Oriented Software" in 1994. It documented 23 recurring solutions to common design problems.

Patterns are not code to copy — they're vocabulary. Saying "we need a Factory here" conveys design intent quickly to any experienced developer.

Important: Don't force patterns everywhere. Overusing patterns is called "patternitis" — it makes simple problems complicated. Use a pattern when you're actually solving the problem it's designed for.

8.2 Singleton (Creational)

Singleton — One Instance, Global Access



```

class Logger {
    private static Logger instance;
    private Logger() {} // prevents 'new Logger()'
    public static synchronized Logger getInstance() {
        if (instance == null) instance = new Logger();
        return instance;
    }
}

```

Use for: config, logger, cache, thread pool

Beware: hidden global state, hard to unit-test, thread-safety issues

Figure: Singleton — one instance for the whole program

Intent: Ensure a class has only ONE instance, and provide a global point of access to it.

How:

- Make the constructor private — prevents new.
- Store the one instance in a static field.
- Provide a static method to get it (creating it lazily on first call).

Thread Safety Variants

- **Basic (not thread-safe):** Two threads calling `getInstance()` simultaneously can both see `instance == null` and create two objects.
- **Synchronized method:** Add `synchronized` to `getInstance()`. Safe but slow — every call locks.
- **Double-checked locking:** Check null before AND after synchronized block. Fast in the common case; requires `volatile` to work correctly.
- **Bill Pugh (holder idiom):** A static inner class holds the instance. JVM guarantees the class is initialized once, thread-safely, on first reference. Clean and correct.
- **Enum singleton:** `enum Singleton { INSTANCE; }`. Serialization-safe, reflection-safe, thread-safe. Effective Java's recommended approach.

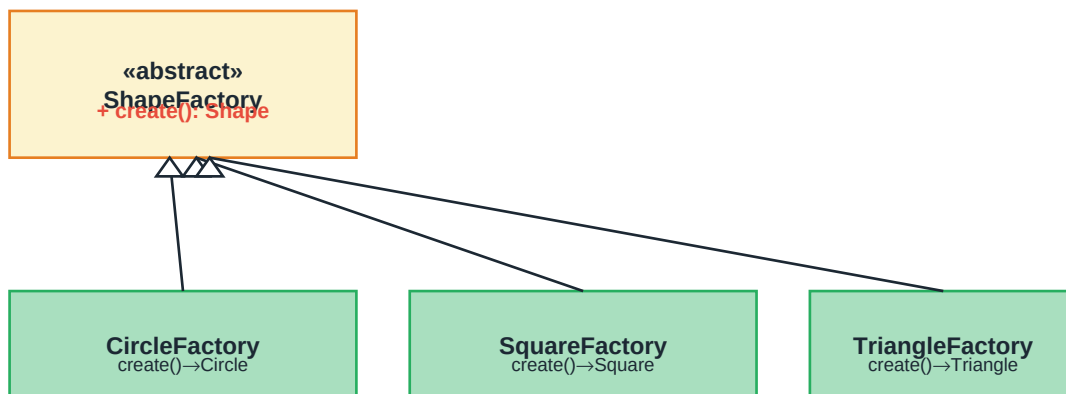
Uses and Abuses

Good uses: logger, configuration, cache, thread pool, connection pool.

Important: Singletons are essentially global variables. They make unit testing hard (can't reset), hide dependencies (methods don't declare using them), and introduce hidden coupling. Some people call Singleton an anti-pattern. Use with restraint.

8.3 Factory Method (Creational)

Factory Method — Delegate Object Creation



Client says: factory.create() — doesn't know or care which concrete type comes back

Use when: need to defer object creation to subclasses; hide instantiation logic

Figure: Factory Method — decouple creation from client code

Intent: Define an interface for creating an object, but let subclasses decide which class to instantiate. Client depends on the factory abstraction, not the concrete product classes.

When to use: When creation logic is complex, when you want to hide which concrete class is used, or when creation depends on configuration.

```

abstract class LoggerFactory {
    public abstract Logger createLogger();
}

class FileLoggerFactory extends LoggerFactory {
    public Logger createLogger() { return new FileLogger(); }
}

class ConsoleLoggerFactory extends LoggerFactory {
    public Logger createLogger() { return new ConsoleLogger(); }
}
  
```

Abstract Factory

One step further: a factory of related factories. Used when you have FAMILIES of related products. E.g., WidgetFactory that creates matching Buttons, Menus, and Scrollbars for a given OS (WindowsFactory, MacFactory, LinuxFactory).

8.4 Builder (Creational)

Builder — Step-by-Step Construction

```
// Instead of huge constructor:  
Pizza p = new Pizza('L', true, true, false, false, 'thin', ...);  
  
// Fluent builder:  
Pizza p = new PizzaBuilder()  
    .size('L').cheese().pepperoni().thinCrust()  
    .build();
```

Solves the 'telescoping constructor' anti-pattern

Each setter returns 'this' → chainable, readable

Use for: complex objects with many optional parameters

Examples: StringBuilder, HttpRequest.Builder, Lombok's @Builder

Figure: Builder — construct complex objects step by step

Intent: Separate construction of a complex object from its representation. Allows step-by-step construction with a fluent API.

Problem it solves: The **telescoping constructor anti-pattern** — when a class has many optional parameters and you end up with a dozen overloaded constructors.

```
// Without builder – hard to read, easy to mess up parameter order  
Pizza p = new Pizza('L', true, false, true, false, 'thin');  
  
// With builder – self-documenting  
Pizza p = new Pizza.Builder()  
    .size('L')  
    .cheese()  
    .pepperoni()  
    .thinCrust()  
    .build();
```

Real-world examples: Java's `StringBuilder`, `HttpRequest.newBuilder()`, Lombok's `@Builder`, Kotlin's DSL builders.

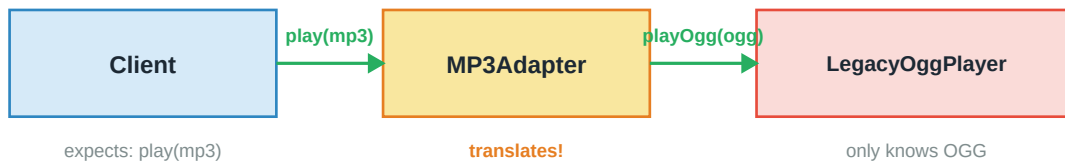
8.5 Prototype (Creational)

Intent: Create new objects by cloning an existing one rather than instantiating a class from scratch. Useful when construction is expensive or the class isn't known at compile time.

Java's `Object.clone()` and `Cloneable` interface implement this. But use them carefully — they default to shallow copy, and `Object.clone()` has famously confusing rules. Consider copy constructors or serialization-based cloning instead.

8.6 Adapter (Structural)

Adapter — Bridge Two Incompatible Interfaces



Convert one interface to another the client expects

Use for: integrating legacy code, matching third-party libraries

Real: Java InputStreamReader (adapts InputStream (bytes) → Reader (chars))

Similar patterns: Facade (simplify), Bridge (decouple), Proxy (control access)

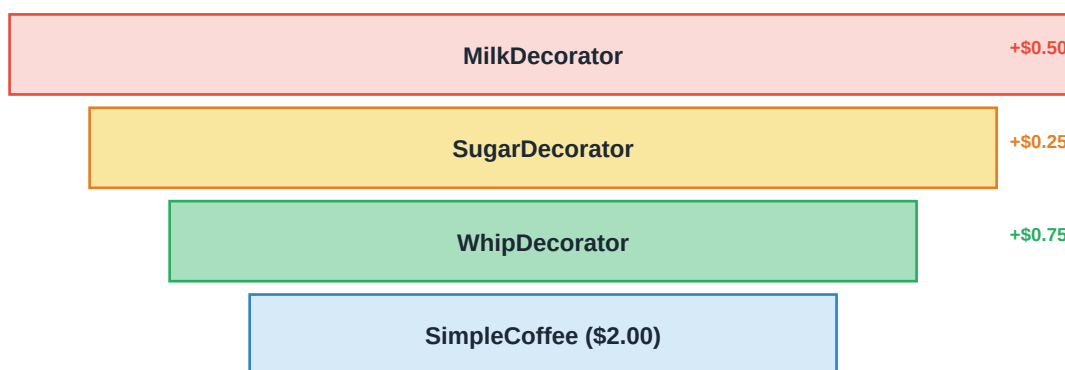
Figure: Adapter — bridge incompatible interfaces

Intent: Convert the interface of a class into another interface the client expects. Adapters let classes work together that couldn't otherwise because of incompatible interfaces.

Real-world examples: Java's InputStreamReader adapts an InputStream (byte-based) to a Reader (character-based). Slack-to-Teams adapters. Physical USB-to-HDMI adapters — same concept.

8.7 Decorator (Structural)

Decorator — Add Behavior Dynamically



coffee.cost() → recursive: milk(sugar(whip(simple))) = \$3.50

Compose behaviors at runtime without exploding class count

Figure: Decorator — layer behavior dynamically

Intent: Add responsibilities to objects DYNAMICALLY without modifying their code. A decorator wraps an object, adds its own behavior, and delegates the rest.

Advantage over inheritance: Can add MULTIPLE independent behaviors and combine them freely. Doesn't cause a subclass explosion (imagine CoffeeWithMilk, CoffeeWithSugar, CoffeeWithMilkAndSugar,

CoffeeWithMilkSugarAndWhip...).

Real-world: Java I/O streams — `new BufferedReader(new InputStreamReader(new FileInputStream(...)))`. Each layer adds behavior.

8.8 Facade (Structural)

Intent: Provide a simplified interface to a complex subsystem. Clients talk to the facade; the facade delegates to the underlying components.

Example: A `HomeTheaterFacade` with `watchMovie()` that internally starts the projector, dims the lights, powers the amplifier, starts the streaming device — clients don't need to know the sequence.

vs Adapter: Adapter changes the interface for one thing; Facade simplifies a group of things.

8.9 Proxy (Structural)

Intent: Provide a surrogate or placeholder for another object to control access to it. The proxy has the same interface as the real object but adds behavior around each call.

Types:

- **Virtual proxy:** Delays creating the expensive real object until actually needed (lazy loading). Hibernate uses this for lazy entity loading.
- **Protection proxy:** Adds access control. "Only admins can call this method."
- **Remote proxy:** Local stand-in for an object on another machine. RPC frameworks use this.
- **Smart proxy:** Adds bookkeeping — reference counting, caching, logging.

8.10 Composite (Structural)

Intent: Compose objects into tree structures to represent part-whole hierarchies. Clients treat individual objects and compositions uniformly.

Example: A file system. A `File` and a `Folder` both implement `Node` with `size()`. `Folder.size()` sums the sizes of its children (which may be `Files` or other `Folders`). Client code just calls `node.size()` without caring whether it's a file or a whole directory tree.

8.11 Strategy (Behavioral)

Strategy — Interchangeable Algorithms

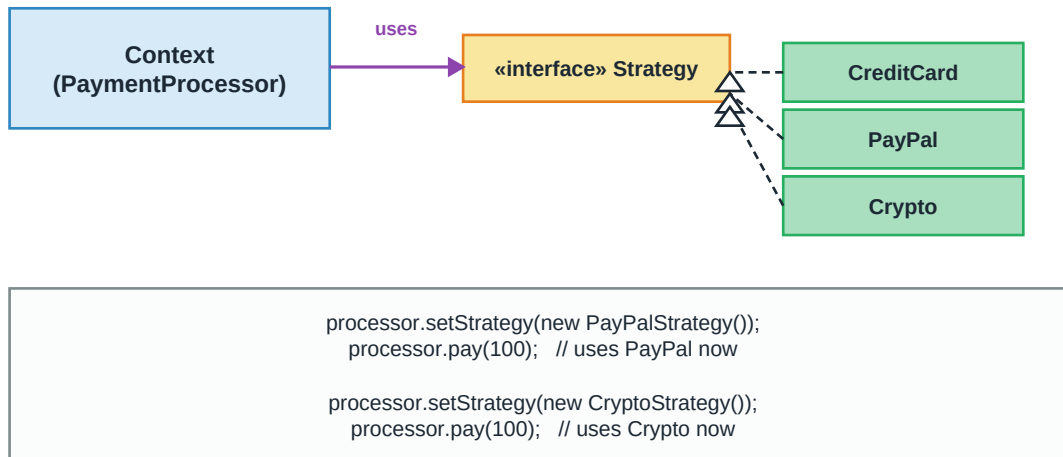


Figure: Strategy — swap algorithms at runtime

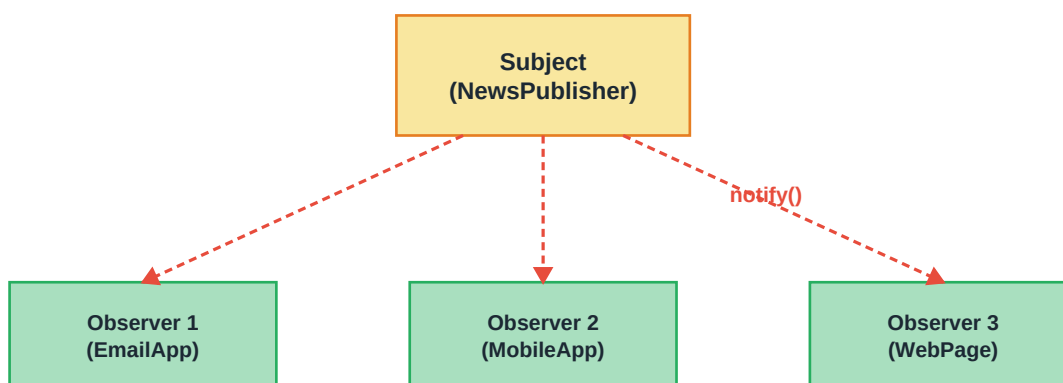
Intent: Define a family of algorithms, encapsulate each, and make them interchangeable. The client picks which strategy to use.

Example: PaymentProcessor takes a PaymentStrategy. CreditCardStrategy, PayPalStrategy, CryptoStrategy all implement it. To switch payment methods, just inject a different strategy — no if-else chain.

This is one of the most useful patterns. Common signs you need Strategy: giant if-else chain or switch statement selecting algorithms, or subclassing just to change one method's behavior.

8.12 Observer (Behavioral)

Observer — Broadcast State Changes



Subject maintains a list of observers; on state change, calls notify() on each

Also called pub/sub, event listener. Foundation of GUI callbacks, RxJava, Redux

Figure: Observer — pub/sub, event notification

Intent: Define a one-to-many dependency so that when one object (subject) changes state, all its dependents (observers) are notified automatically.

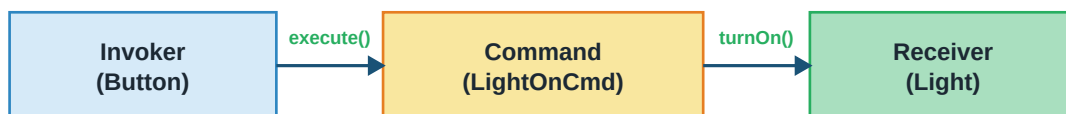
Examples: GUI event listeners, RxJava streams, Redux subscribers, YouTube channel notifications, most modern reactive frameworks.

```
interface Observer { void update(Event e); }

class Subject {
    private List<Observer> observers = new ArrayList<>();
    public void subscribe(Observer o) { observers.add(o); }
    public void unsubscribe(Observer o) { observers.remove(o); }
    public void notifyAll(Event e) {
        for (Observer o : observers) o.update(e);
    }
}
```

8.13 Command (Behavioral)

Command — Encapsulate a Request as an Object



Enables: undo/redo, queuing, logging, macro commands

Real: java.lang.Runnable, GUI event handlers

Every button press packages what to do → decouples caller from callee

cmdHistory.push(cmd); cmd.execute(); // to undo: cmdHistory.pop().undo();

Figure: Command — request as an object

Intent: Encapsulate a request as an object. Lets you parameterize clients with different requests, queue or log requests, and support undo.

Example: A text editor's undo/redo. Every action (typing, deleting, formatting) becomes a Command object with execute() and undo() methods. The editor keeps a stack of commands. Undo? Pop the stack and call undo() on it.

Real: Runnable in Java (a task to execute), GUI event handlers, database transactions.

8.14 Template Method (Behavioral)

Intent: Define the skeleton of an algorithm, letting subclasses override specific steps. The overall structure is fixed; the specific steps vary.

```
abstract class Beverage {
    public final void prepare() { // template, don't override
        boilWater();
        brew(); // specific to each subclass
        pourInCup();
        addCondiments(); // specific to each subclass
    }
    protected abstract void brew();
    protected abstract void addCondiments();
    private void boilWater() { ... }
    private void pourInCup() { ... }
}
```

Tea and Coffee subclasses implement `brew()` and `addCondiments()` differently. The overall sequence is guaranteed.

8.15 Iterator (Behavioral)

Intent: Provide a way to sequentially access elements of a collection without exposing its underlying representation. So common it's built into most languages.

Every language's `for-each` is powered by the Iterator pattern. Java's `Iterator` interface. Python's `__iter__`. Ruby's `each`.

8.16 State (Behavioral)

Intent: Allow an object to alter its behavior when its internal state changes. The object APPEARS to change class.

Example: A vending machine has states: `NoCoin`, `HasCoin`, `Dispensing`, `OutOfStock`. Actions (`insertCoin`, `selectProduct`) do different things in each state. Instead of nested if-else, define a `State` interface with these actions, and have concrete state classes. The machine holds a current state; transitions swap the current state.

Key Insight: State pattern is the OOP answer to the finite state machine. Great for complex workflows: order processing (`Placed` → `Paid` → `Shipped` → `Delivered`), authentication, game character behavior.

8.17 Chain of Responsibility (Behavioral)

Intent: Pass a request along a chain of handlers. Each handler decides to either process it or pass it to the next handler.

Examples: HTTP middleware chains (Express, Spring interceptors), event bubbling in GUIs, servlet filters, exception handlers going up the call stack.

Chapter 9

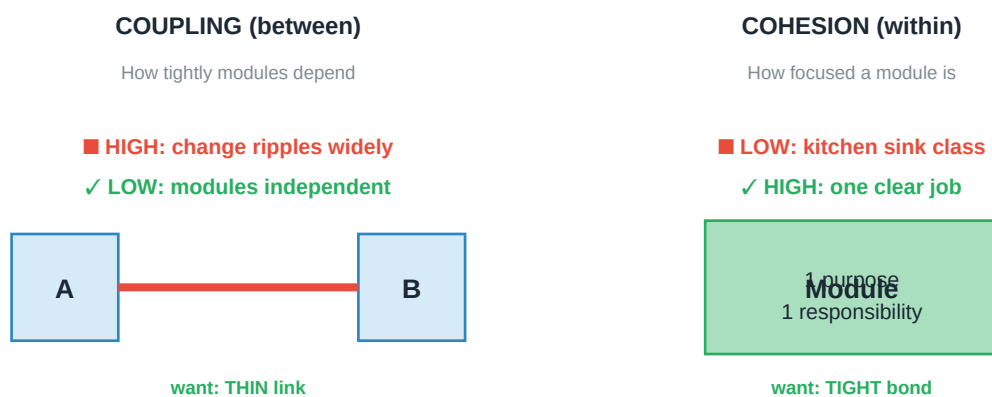
Advanced OOP Concepts

By this point you know the four pillars, SOLID, and the main design patterns. This chapter covers the harder, subtler topics that separate senior developers from juniors — the things that come up in system design and hard interviews.

9.1 Coupling and Cohesion

Two metrics that quietly determine whether a codebase ages gracefully or turns into unmaintainable mud. Every principle you've learned so far (SOLID, encapsulation, composition-over-inheritance) is ultimately about improving these two.

Coupling and Cohesion — Two Fundamental Quality Metrics



Goal: LOW coupling + HIGH cohesion. Every design principle helps you get there.

Figure: Coupling (between modules) vs Cohesion (within a module)

Coupling — The Wire Between Modules

How tightly two modules depend on each other. Change one, and how much does the other need to change too? Levels from worst to best:

- **Content coupling (worst):** One module modifies internal data of another. Almost extinct in modern languages that enforce encapsulation.
- **Common coupling:** Modules share global data. Change to the format breaks both.
- **Control coupling:** One module passes flags to control the other's flow — `processOrder(order, true, false)`. What do those booleans mean?
- **Stamp coupling:** Modules share a complex data structure but each uses only part of it.
- **Data coupling (good):** Modules communicate via simple parameters. Change one; others usually unaffected.
- **Message coupling (best):** Communication only through well-defined interfaces (methods).

Cohesion — The Focus Within a Module

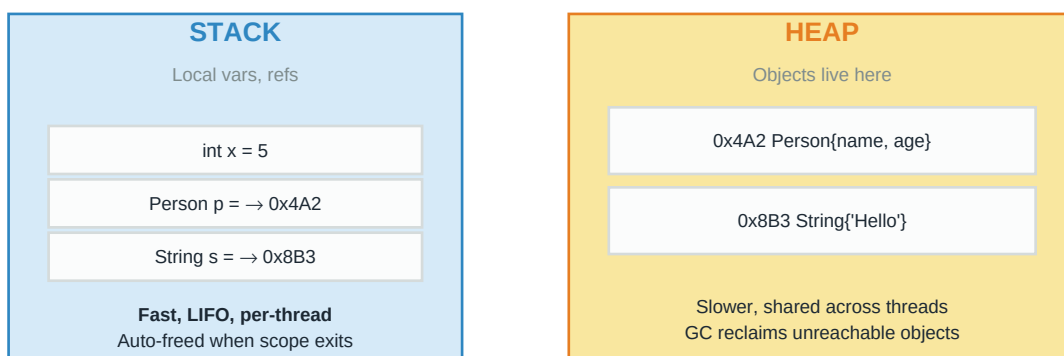
How well the responsibilities of a module go together. High cohesion = the module does ONE thing well. Low cohesion = it's a random collection of unrelated stuff.

- **Coincidental (worst):** Parts grouped for no clear reason. "Utils.java" that has methods for string manipulation, date formatting, and network requests.
- **Logical:** Parts do similar things but for different data types.
- **Temporal:** Parts execute at the same time ("initialization" module doing 20 unrelated setup tasks).
- **Procedural:** Parts contribute to the same overall procedure.
- **Sequential:** Output of one part feeds into the next.
- **Functional (best):** All parts contribute to a single, well-defined task. UserAuthenticator does authentication and nothing else.

Key Insight: Rule for life: **LOW coupling, HIGH cohesion**. Every architectural decision should nudge toward this. SRP directly improves cohesion. DIP directly reduces coupling.

9.2 Object Memory — Stack vs Heap

Object Memory — Stack vs Heap



In Java/C#: all objects on heap; local variables and references on stack.

In C++: choice yours (new = heap, plain declaration = stack).

Figure: Objects usually live on the heap; references to them on the stack

The Stack

Each thread has its own stack — fast, LIFO memory used for local variables and method call frames. When a method is called, a frame is pushed with its locals and parameters. When it returns, the frame is popped, and its memory is instantly freed. No GC involved.

In Java, C#, Python, everything you "see" on the stack is either a primitive (int, char, boolean) or a REFERENCE to an object. The objects themselves live elsewhere.

The Heap

A large shared memory region where objects live. Slower access than stack, but flexible — objects can outlive the method that created them. Shared across all threads (which is why concurrent access needs synchronization).

In Java/C#/Python, EVERY object is on the heap. Local variables in methods just hold references (essentially pointers) to those heap objects. In C++, you choose: `Point p;` puts it on the stack, `Point* p = new Point();` puts it on the heap.

Garbage Collection

Objects on the heap don't clean themselves up. In C++, YOU delete them (or use smart pointers). In Java/C#/Python, a **garbage collector (GC)** automatically reclaims memory when no references remain.

- **Reference counting (Python's primary strategy):** Each object tracks how many references point to it. When count hits zero → deallocate. Simple, but can't handle cycles (A points to B, B points to A, both unreachable but count stays >0). Python has a cycle detector as backup.
- **Mark-and-sweep (Java, Go, C#):** Periodically, traverse from root references (stack, static fields), mark all reachable objects. Then sweep unmarked ones. Handles cycles perfectly.
- **Generational GC:** Observation — most objects die young. So separate young and old generations. Collect young frequently (fast), old rarely. JVM does this.

Important: GC doesn't eliminate memory leaks — it just changes what a "leak" means. If you accumulate references in a long-lived collection (a cache, a listener list), those objects stay alive forever. "I forgot to unsubscribe" is a classic Java memory leak.

9.3 Shallow Copy vs Deep Copy

Shallow Copy vs Deep Copy

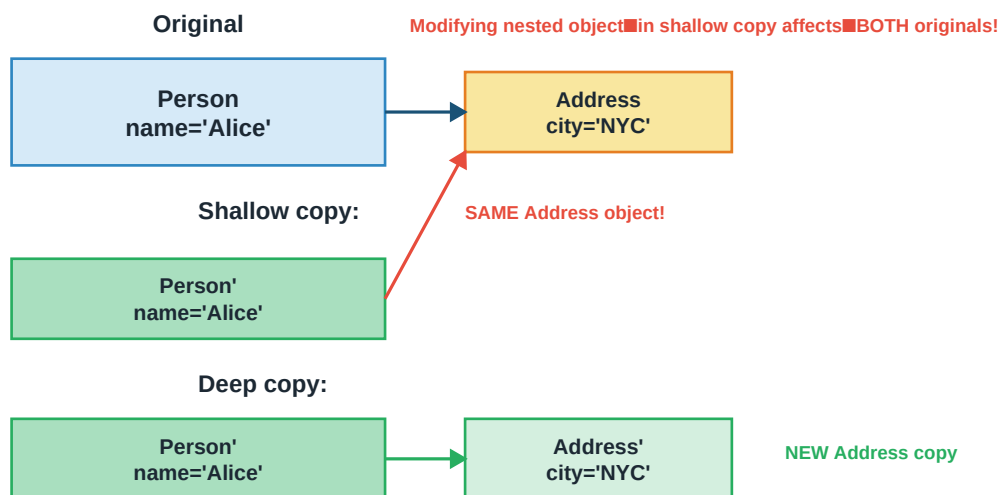


Figure: Shallow copy shares nested objects; deep copy duplicates everything

When you copy an object with nested objects, the depth of the copy matters.

- **Shallow copy:** Duplicates the outer object but shares its inner references. If the original and copy share a nested list, modifying the list through one affects the other.
- **Deep copy:** Recursively duplicates the entire object graph. Original and copy are truly independent.

Language Mechanisms

- **Java:** `Object.clone()` defaults to shallow copy. Deep copy requires overriding it, using serialization, or writing a copy constructor. `Cloneable` interface has famously confusing semantics.
- **Python:** `copy.copy(obj)` is shallow, `copy.deepcopy(obj)` is deep.
- **C++:** Default copy constructor is shallow. If your class holds raw pointers, you MUST write a custom copy constructor for deep copy. "Rule of Three" (now Rule of Five).
- **JavaScript:** `Object.assign()` and spread `{...obj}` are shallow. Deep clone: `structuredClone(obj)` or `JSON.parse(JSON.stringify(obj))` (limited).

Key Insight: Whenever an object has mutable nested objects, decide explicitly which copy semantics you want. Shallow-by-default has caused countless bugs.

9.4 Object Equality vs Identity

Object Identity vs Equality

```
String a = new String('hello');
String b = new String('hello');

a == b    → false (different objects in memory, IDENTITY)
a.equals(b) → true  (same content, EQUALITY)

== compares references (are these the SAME object?)
.equals() compares logical values (do these represent the SAME thing?)

Golden rule: override .equals() → also override .hashCode()
(else hash collections like HashSet/HashMap will misbehave)
```

Figure: Two crucial comparisons that people confuse

Identity: are these two references pointing to the SAME object in memory? `==` in Java (for objects), `is` in Python.

Equality: do these two objects REPRESENT the same value? `.equals()` in Java, `==` in Python, `Equals()` in C#.

The equals/hashCode Contract

Interview classic. In Java (and most OO languages with hash-based collections), if you override `equals()`, you MUST also override `hashCode()`. The contract:

- If `a.equals(b)` is true, then `a.hashCode()` must equal `b.hashCode()`.
- If hashCodes are equal, equals doesn't HAVE to be true (collisions are allowed).
- `hashCode` should be consistent — same object, same value, every call.

Important: Break this contract and `HashMap/HashSet` silently misbehave. You put an object in; you can't find it again — because `HashMap` put it in one bucket based on `hashCode`, then looks in a DIFFERENT bucket when you search. Classic silent bug.

```
@Override
public boolean equals(Object o) {
    if (this == o) return true;
```

```

if (!(o instanceof Person)) return false;
Person p = (Person) o;
return this.id == p.id;
}

@Override
public int hashCode() {
    return Objects.hash(id); // matches equals – uses same fields
}

```

Key Insight: Modern Java: record types auto-generate correct equals/hashCode. Kotlin's `data class` does the same. Lombok's `@Data` too. Use these where possible.

9.5 Generics and Templates

Generics let you write code that works with ANY type, while keeping type safety. Before generics (Java 1.4 and earlier), a `List` stored `Objects` — you had to cast on retrieval, and mixing types would blow up at runtime.

```

// Without generics – no compile-time safety
List list = new ArrayList();
list.add('hello');
list.add(42); // oops, mixed types
String s = (String) list.get(1); // ClassCastException at runtime

// With generics
List<String> list = new ArrayList<>();
list.add('hello');
list.add(42); // COMPILE ERROR
String s = list.get(0); // no cast needed

```

Bounded Generics

You can constrain what types a generic accepts. `<T extends Number>` means `T` must be a `Number` or subclass. Useful for methods that need to call methods available on `Number`.

Java Type Erasure (Interview Trap)

Important: Java generics use **type erasure** — generic type info is REMOVED at compile time. At runtime, `List<String>` and `List<Integer>` are both just `List`. Consequences: can't do `new T()`, can't check `obj instanceof List<String>`, can't have overloaded methods that differ only in generic type parameters.

C# and Kotlin (JVM) keep generic type info at runtime (reified generics) — more powerful, at some cost. C++ templates are actually different: each template instantiation generates separate code at compile time (monomorphization). Much faster at runtime, but code bloat.

9.6 Reflection and Metaprogramming

Reflection is the ability of a program to inspect and modify its own structure at runtime. You can list a class's methods and fields, invoke methods by name, create objects without knowing the class at compile time, and even change access modifiers.

```
// Java reflection example
Class<?> cls = Class.forName('com.example.Person');
Object obj = cls.getConstructor(String.class).newInstance('Alice');
Method m = cls.getMethod('greet');
m.invoke(obj); // calls person.greet()
```

Where Reflection is Used

- **Frameworks:** Spring, Hibernate, JUnit — they read your annotations, instantiate your classes, and wire everything together. Impossible without reflection.
- **Serialization libraries:** Jackson, Gson use reflection to read/write your fields.
- **Testing tools:** Mocking libraries (Mockito) generate proxies at runtime.
- **ORMs:** Map database columns to object fields dynamically.

Important: Reflection is **slow** (much slower than direct calls), breaks type safety, and can bypass encapsulation. Use it when necessary (framework code), avoid it in application code.

9.7 Method Resolution Order and Virtual Inheritance

Python's MRO

Python allows multiple inheritance. When you call a method, in which order does Python search parent classes? Uses **C3 linearization** — a deterministic algorithm producing the Method Resolution Order (MRO).

```
class A: def m(self): print('A')
class B(A): def m(self): print('B')
class C(A): def m(self): print('C')
class D(B, C): pass

D().m() # prints 'B' — B comes before C in MRO
D.__mro__ # (D, B, C, A, object)
```

Rules C3 enforces: children before parents, order of parents preserved, monotonic (a class's MRO is a subsequence of every child's MRO). If no valid linearization exists, Python raises `TypeError` at class creation.

C++ Virtual Inheritance

C++ diamond problem produces **DUPLICATE** base state — `SmartPhone` would have **TWO** `Device` sub-objects. **Virtual inheritance** fixes this: mark inheritance as virtual, and there's only one shared `Device` sub-object.

```
class Device { ... };
class Phone : virtual public Device { ... };
class Camera : virtual public Device { ... };
class SmartPhone : public Phone, public Camera { ... };
// One Device in SmartPhone, shared between Phone and Camera
```

9.8 Composition Deep Dive — Aggregation vs Composition

"Composition" in casual speech means HAS-A. In UML, there are two flavors of HAS-A:

- **Aggregation:** The parts CAN exist independently of the whole. A university has students, but if the university closes, the students still exist. UML: hollow diamond.
- **Composition:** The parts CANNOT exist without the whole. A house has rooms; if you demolish the house, the rooms are gone too. UML: filled diamond.

Composition implies LIFECYCLE MANAGEMENT — the containing object is responsible for creating and destroying its parts. Aggregation is a weaker "knows about" relationship.

Chapter 10

Interview Patterns & Common Questions

Everything so far is the theory. This chapter is the payoff — how OOP interviews actually go, what gets asked over and over, and how to answer with confidence. The last section maps out the LLD problems that come up in nearly every OOP-heavy interview.

10.1 The OOP Interview Landscape

OOP interviews at product/service companies generally test four things:

- **Conceptual clarity:** Can you explain the four pillars with real examples? Do you know the difference between abstraction and encapsulation?
- **Code fluency:** Given a small problem, can you write clean OO code — right classes, right responsibilities, right coupling?
- **Design principles:** SOLID, DRY, KISS, YAGNI, Law of Demeter, composition over inheritance. Do you know when to apply them?
- **Low-Level Design (LLD):** Design a Parking Lot. Elevator. ATM. etc. from scratch in 45 minutes.

Key Insight: Interviewers value CLARITY over cleverness. Explaining your thought process is worth more than a perfect answer given silently. Talk through decisions: "I'm making this abstract because I expect future extension", "I'm using composition here because..."

10.2 Most-Asked Conceptual Questions & Ideal Answers

Q: What is OOP?

OOP is a programming paradigm that organizes code around **objects** — self-contained units that bundle data (state) with the operations (behavior) that act on that data. It rests on four pillars: encapsulation, inheritance, polymorphism, and abstraction. The core idea is to model software the way we naturally think about the real world — as interacting entities rather than a sequence of steps.

Q: Explain the four pillars with a real example

Take a BankAccount class:

- **Encapsulation:** balance is private; you interact only via deposit(), withdraw(), getBalance(). The account can enforce rules like "balance can never go negative".
- **Inheritance:** SavingsAccount and CurrentAccount extend BankAccount, inheriting shared behavior while specializing (SavingsAccount adds interest calculation).
- **Polymorphism:** A method that takes an Account can work with either SavingsAccount or CurrentAccount. Calling account.calculateInterest() runs the right version at runtime.

- **Abstraction:** Clients see a clean interface (deposit, withdraw, calculateInterest) without knowing how the underlying ledger, transactions, or interest formulas work.

Q: Abstract class vs Interface — when do you pick which?

Use an **abstract class** when subclasses share significant common CODE and state, and there's a genuine IS-A relationship. Use an **interface** when you're defining a capability that unrelated classes should implement (Comparable, Serializable, Iterable), or when multiple inheritance of type is needed. Modern rule of thumb: default to interfaces. Java 8+ default methods let interfaces carry behavior, closing much of the historical gap.

Q: Overloading vs Overriding?

Overloading = same method name, DIFFERENT parameters, SAME class. Resolved at COMPILE time. **Overriding** = same method name, SAME parameters, DIFFERENT classes (parent → child). Resolved at RUNTIME by actual object type. Overloading is compile-time polymorphism; overriding is runtime polymorphism. Overriding is what makes dynamic dispatch and "one call, many behaviors" work.

Q: Static vs Instance members?

Instance members belong to each object — every object has its own copy. **Static** members belong to the class itself — one copy shared by all objects. Static methods can't access instance fields (no this reference). Use static for utility functions, constants, and counters. Avoid it for state that has any real complexity — it's essentially a global variable, with all the problems that implies.

Q: Composition vs Inheritance — which do you prefer and why?

Composition, in most cases. Inheritance creates a rigid, tightly-coupled relationship — the child depends not just on the parent's PUBLIC interface but on its INTERNAL BEHAVIOR (fragile base class problem). Composition is more flexible: swap out components, combine multiple behaviors, avoid the diamond problem. Effective Java Item 18: prefer composition over inheritance. Reach for inheritance only when there's a clean IS-A relationship AND the parent was designed for extension.

Q: When would you use a Singleton?

When you genuinely need exactly ONE instance for the whole application AND global access to it — loggers, configuration, connection pools, caches. But be honest with yourself: often "we need one instance" is just laziness about dependency management. Singletons are essentially global state — they hide dependencies, make testing hard, and cause thread-safety problems. Modern advice: prefer dependency injection, use singleton scope in your DI container instead of the pattern by hand.

Q: How do you make a Singleton thread-safe?

Options in order of quality: (1) **Enum singleton** — Java's cleanest, thread-safe by JVM guarantee, serialization-safe. (2) **Bill Pugh's holder idiom** — static inner class holds the instance, JVM handles thread-safe initialization. (3) **Double-checked locking** with volatile — old-school, works but tricky. (4) **Synchronized getInstance()** — simple but slow, every call locks.

Q: Why prefer interfaces to concrete classes in method parameters?

Loose coupling. If a method takes `List<String>`, it works with `ArrayList`, `LinkedList`, or any custom `List`. If it takes `ArrayList<String>`, you can only pass `ArrayList` — and you've locked callers to that implementation forever. "Program to an interface, not an implementation" is one of the GoF's fundamental principles.

Q: What's the diamond problem and how does Java solve it?

In multiple inheritance, if a class inherits from two parents that both inherit from a common grandparent, there's ambiguity about which grandparent version to use. Java sidesteps this by disallowing multiple class inheritance — you can only extend ONE class. Multiple inheritance of TYPE is allowed through interfaces (which historically had no implementation). With Java 8 default methods, if two interfaces provide conflicting defaults, the implementing class MUST explicitly override to resolve.

Q: Why should you override equals() and hashCode() together?

Because hash-based collections (HashMap, HashSet) rely on the CONTRACT that equal objects have equal hashCodes. HashMap uses hashCode to pick a bucket, then equals to find the exact entry within that bucket. If you override equals but not hashCode, two "equal" objects can land in different buckets — you put one in and can never find it again. Silent, painful bug. Always override them together, using the same fields.

Q: Why use a final class?

Three reasons: (1) **Security** — String is final so no one can subclass it and break invariants (immutability guarantees, hashCode stability). (2) **Design intent** — you're saying "this class is complete, not meant for extension", closing off fragile base class risks. (3) **Performance** — JVM can inline final methods more aggressively. Effective Java: design and document for inheritance or else prohibit it (make it final).

Q: Why private constructors?

To control instantiation. Uses: (1) **Singleton pattern** — no external new. (2) **Utility classes** — Math, Collections have no instances; private constructor prevents accidental new Math(). (3) **Static factory methods** — force clients to use Integer.valueOf(5) instead of new Integer(5), enabling caching. (4) **Builder pattern** — the outer class's constructor is private; only the builder can call it.

Q: Method hiding vs Overriding for static methods?

Static methods can't be OVERRIDDEN — they're bound to the class, not the instance. But if a subclass declares a static method with the same signature as the parent's, it HIDES the parent's version. The version called depends on the REFERENCE TYPE, not the object type. This is a common trick question: Parent p = new Child(); p.staticMethod(); calls the PARENT's version because the reference is Parent — the opposite of instance methods.

Q: What is dependency injection and why use it?

DI is a technique where a class's dependencies are provided by an external caller instead of being created internally. Constructor injection is most common: instead of new MySQLDatabase() inside OrderService, OrderService takes a Database in its constructor and stores it. Benefits: testability (inject mocks), flexibility (swap implementations), explicit dependencies (no hidden coupling), and enables the Dependency Inversion Principle. Frameworks like Spring automate the wiring.

10.3 Code Smells Interviewers Look For

Smells are surface signs that something deeper is wrong. Naming them shows OOP maturity.

- **God class:** One massive class doing everything. Break by SRP.

- **Long method:** A method that scrolls off the screen. Extract sub-methods.
- **Feature envy:** A method uses another class's data more than its own. Move the method there.
- **Data class:** A class with only fields and getters/setters, no behavior. Ask: where does the behavior on this data live? It should live HERE.
- **Shotgun surgery:** One conceptual change requires editing many classes. Points to poor encapsulation.
- **Parallel inheritance hierarchies:** Every time you add a subclass to hierarchy A, you must add one to hierarchy B. Symptom of tangled design.
- **Refused bequest:** A subclass throws away inherited methods ("my parent has printSlip() but I don't need it — throw NotImplementedException"). LSP violation.
- **Switch statements on type:** Big if-else or switch on an object's type. Missing polymorphism.
- **Primitive obsession:** Passing raw strings and ints everywhere instead of little wrapper types. Address, Money, Email should be types, not strings.

10.4 Language-Specific Gotchas

Java Gotchas

- **Autoboxing performance:** `Long sum = 0L; for (long i=0; i<n; i++) sum += i;` — sum is boxed, unboxed, re-boxed each iteration. Massive slowdown.
- **Integer cache:** `Integer.valueOf(100) == Integer.valueOf(100)` is true (cached -128 to 127), but for 200 it's false. Always use `.equals()`.
- **String comparison:** `==` compares references; `.equals()` compares content. String literals are interned; `new String("hi")` creates a new object.
- **Generics type erasure:** `List<String>` and `List<Integer>` are the same class at runtime. Can't do `new T()`.
- **Static + inheritance:** Static methods aren't overridden — they're hidden. `Parent p = new Child(); p.staticMethod()` calls Parent's.

C++ Gotchas

- **Missing virtual destructor:** If a base class isn't virtual, `delete basePtr` won't call the derived destructor. Memory leak.
- **Slicing:** Passing a Derived object by value to a function expecting Base slices off the derived parts.
- **Rule of Three/Five/Zero:** If you define ONE of destructor, copy constructor, or copy assignment, you almost certainly need all THREE. C++11 adds move constructor and move assignment (Five).
- **Non-virtual method by base pointer:** No polymorphism — calls the BASE version even if the object is Derived.

Python Gotchas

- **Mutable default arguments:** `def f(x, arr=[])` — the SAME list is reused across every call. Classic bug. Use `None` and create inside.
- **Class attributes vs instance attributes:** If you don't assign to self, you might modify the class attribute (shared across all instances). Bites you with mutable class attributes.
- **No true privacy:** `_x` is convention only. `__x` triggers name mangling but is still accessible.
- **MRO surprises:** Multiple inheritance can produce non-obvious method resolution.

10.5 The LLD Interview — A Template Approach

Low-Level Design questions ("design a parking lot", "design an elevator system") test whether you can go from a fuzzy problem statement to clean OO code. Follow this five-step template.

Step 1 — Clarify Requirements (5 min)

Don't jump to code. Ask clarifying questions. For a parking lot: How many floors? Different spot sizes? Different vehicle types (bike, car, truck)? Payment methods? Reservation? Multiple entry points? Handicapped spots? This shapes the whole design.

Step 2 — Identify Entities & Actors (5 min)

Name the nouns from the problem. For a parking lot: ParkingLot, Floor, ParkingSpot, Vehicle, Ticket, Payment, Gate, Attendant. Not every noun becomes a class — but this gives you the pieces to work with.

Step 3 — Define Relationships (10 min)

How do the entities relate? ParkingLot has-many Floors. Floor has-many ParkingSpots. Vehicle is-a Car / Bike / Truck (inheritance). ParkingSpot is-a CarSpot / BikeSpot / TruckSpot. Ticket references a Vehicle and a ParkingSpot. Draw it as a rough class diagram.

Step 4 — Apply Design Patterns (10 min)

Where do patterns naturally fit? For a parking lot: **Strategy** for pricing (hourly, flat rate, discount). **Factory** to create the right Ticket type. **Singleton** for the ParkingLot itself (usually just one). **State** for a spot's status (Free, Reserved, Occupied). Only use a pattern if it genuinely fits — don't force it.

Step 5 — Code the Core Classes (15 min)

Write the SKELETON code — classes, fields, method signatures, key relationships. Focus on the "happy path" of one main scenario (parkVehicle, exitVehicle). No need to implement every method body. Talk through decisions as you go. Interviewer will drill into specific parts.

Key Insight: SOLID principles matter here. Interviewers actively look for SRP (each class one job), OCP (adding a new VehicleType shouldn't require rewriting existing code), DIP (Payment depends on PaymentStrategy interface, not concrete CreditCardPayment).

10.6 The Most-Asked LLD Problems

OOP Interview Question Map (LLD focus)

Parking Lot	Elevator	ATM	Vending Machine
Composition, State, Factory	State, Strategy, Queue	State, Command, Strategy	State machine
Chess	Snake & Ladder	Splitwise	Ride Sharing
Strategy, Command, MVC	Composition, Iterator	Observer, Strategy	Strategy, Observer
LRU Cache	Logger	Rate Limiter	Notification Sys
Doubly LL + HashMap	Singleton, Chain	Strategy, Bucket	Observer, Strategy

These come up over and over. Practice designing 5-6 of them well.

Figure: The classic LLD questions and the patterns each usually needs

These come up over and over. Practice designing at least 5-6 well. The patterns column shows what usually fits — not what you MUST use.

Parking Lot

Entities: ParkingLot, Floor, ParkingSpot (abstract → CarSpot, BikeSpot, TruckSpot), Vehicle (abstract → Car, Bike, Truck), Ticket, Payment.

Patterns: Factory (create Ticket), Strategy (pricing), Singleton (ParkingLot), State (spot status).

Key challenge: Efficiently finding the nearest available spot of the right size. Consider PriorityQueue keyed by distance.

Elevator System

Entities: Elevator, ElevatorController, Floor, Button (external, internal), Request.

Patterns: Strategy (scheduling algorithm — SCAN, LOOK, FCFS), State (Moving, Idle, Maintenance), Command (each button press = command in queue).

Key challenge: Multi-elevator coordination — which elevator responds to which request?

ATM

Entities: ATM, Card, Account, Transaction (Withdraw, Deposit, BalanceInquiry, Transfer), CashDispenser, Screen, Keypad, BankService.

Patterns: State (Idle, CardInserted, AuthenticationSuccess, Transaction, Ejecting), Command (each transaction type), Strategy (authentication method), Chain of Responsibility (denomination dispensing).

Key challenge: Cash dispensing algorithm — dispensing ₹1650 with limited notes.

Vending Machine

Entities: VendingMachine, Product, Inventory, Coin, State.

Patterns: **State** is the star here — NoCoinState, HasCoinState, DispensingState, OutOfStockState. Each state handles insertCoin(), selectProduct(), refund() differently.

Key challenge: Managing state transitions cleanly.

Chess

Entities: ChessBoard, Cell, Piece (abstract → King, Queen, Rook, Bishop, Knight, Pawn), Player, Move, Game.

Patterns: Strategy (each piece has its own move validation), Command (each Move is a command — enables undo), MVC (separate board state, rules, display).

Key challenge: Move validation — piece-specific rules + board state (check, checkmate).

Splitwise

Entities: User, Group, Expense, Split (abstract → EqualSplit, ExactSplit, PercentSplit), Balance, Transaction.

Patterns: Strategy (split types), Observer (notify users of new expenses), Composite (a group of groups?).

Key challenge: Efficiently computing minimum transactions to settle balances (min-cash-flow). Use a max-heap and min-heap of balances.

LRU Cache

Data structures: HashMap + Doubly Linked List. HashMap gives O(1) lookup; DLL gives O(1) reordering when accessed.

Operations:

get(k): if present, move node to head (most recent), return value.

put(k, v): if present, update and move to head. If new, add to head; if capacity exceeded, remove tail node and its map entry.

Patterns: Often modeled with Decorator (add TTL, add persistence, add metrics as separate wrapper caches).

Logger

Entities: Logger, LogLevel (DEBUG, INFO, WARN, ERROR, FATAL), Appender (Console, File, Network), LogFormatter.

Patterns: Singleton (one logger per name), Chain of Responsibility (each level handler passes up if not applicable), Strategy (Appender), Observer (multiple appenders subscribe to the log stream).

Key challenge: Async logging without dropping messages under load.

Rate Limiter

Algorithms: Token Bucket (allows bursts up to bucket size), Leaky Bucket (smooths rate to constant output), Sliding Window Counter (rolling counter), Fixed Window Counter (simplest).

Patterns: Strategy (pick the algorithm), Decorator (wrap any service with rate limiting).

Key challenge: Distributed rate limiting — coordination across servers, typically via Redis.

Notification System

Entities: User, Notification, Channel (Email, SMS, Push, InApp), Template, PreferenceManager.

Patterns: Observer (users subscribe to events), Strategy (channel-specific formatting), Factory (create right notification type), Chain of Responsibility (try channels in preferred order).

Key challenge: At-least-once delivery, retry with backoff, deduplication.

10.7 Final Interview Cheat Sheet

Before the Interview — Prepare

- Have **ONE crisp story** for each of the four pillars using the same running example.
- Know 6-7 design patterns DEEPLY (Singleton, Factory, Strategy, Observer, Decorator, Adapter, Command).
- Practice designing 5-6 LLD problems end-to-end, timed.
- Know your language's OOP gotchas.
- Be able to explain SOLID with concrete examples — not just the acronym.

During the Interview — Behavior

- Ask clarifying questions BEFORE coding. Assumptions kill.
- Think out loud. Show your reasoning.
- Start with the interface / class skeleton before implementation.
- Justify design choices in SOLID terms.
- When you get stuck: state the problem clearly, then explore options.

Red Flags to Avoid

- Making everything static — screams "I don't get OO".
- Putting all logic in one big class.
- Deep inheritance hierarchies (more than 2-3 levels rarely justified).
- if-else on type instead of polymorphism.
- Silence. Even wrong reasoning out loud beats being quiet.

Analogy: The final secret of OOP interviews: interviewers aren't testing whether you can recite definitions. They're testing whether you can *DESIGN SOFTWARE* — whether they'd trust you to build the next module in their codebase. Every answer should show that you think about code the way a senior developer does: for the humans who'll maintain it, for the changes coming next month, for the correctness under stress. Everything else follows from that.

Appendix

Quick Reference — Everything on One Handy Sheet

The night-before-interview reference. All the essentials in the smallest useful form.

A.1 The Four Pillars — One-Liners

- **Encapsulation:** Bundle data + methods; hide internals behind public interface.
- **Inheritance:** Child class reuses and extends parent's attributes and methods (IS-A).
- **Polymorphism:** One interface, many implementations; runtime picks based on actual type.
- **Abstraction:** Expose essential behavior; hide implementation.

A.2 SOLID Mnemonics

S – Single Responsibility → one reason to change
O – Open / Closed → open to extension, closed to modification
L – Liskov Substitution → subclasses replace parents without breaking
I – Interface Segregation → many small interfaces > one fat one
D – Dependency Inversion → depend on abstractions, not concretes

A.3 Overloading vs Overriding

Aspect	Overloading	Overriding
Where	Same class	Parent → child
Parameters	Must differ	Must match
Return type	Can differ	Same or covariant
Access	Any	Same or wider
Binding	Compile-time (static)	Runtime (dynamic)
Polymorphism	Ad-hoc	Subtype (true OOP)

A.4 Abstract Class vs Interface

Feature	Abstract Class	Interface
Purpose	Partial impl + IS-A	Contract, CAN-DO
Methods	Abstract + concrete	Abstract (+ default in Java 8+)
Fields	Any (incl. state)	public static final only
Constructors	Yes	No

Feature	Abstract Class	Interface
Multiple inheritance	One only	Many
Use when	Common code + variation	Multiple unrelated types

A.5 The 23 GoF Design Patterns

Creational (5):

- Singleton, Factory Method, Abstract Factory, Builder, Prototype

Structural (7):

- Adapter, Bridge, Composite, Decorator, Facade, Flyweight, Proxy

Behavioral (11):

- Chain of Responsibility, Command, Interpreter, Iterator, Mediator, Memento, Observer, State, Strategy, Template Method, Visitor

A.6 Pattern Intents — the top 12

Pattern	One-line Intent
Singleton	One instance, global access
Factory Method	Delegate creation to subclasses
Builder	Step-by-step construction of complex objects
Adapter	Convert one interface into another
Decorator	Add responsibilities dynamically without subclassing
Facade	Simplified interface to a complex subsystem
Proxy	Surrogate to control access to another object
Strategy	Family of interchangeable algorithms
Observer	One-to-many state change notification
Command	Encapsulate a request as an object (enables undo)
Template Method	Skeleton with steps for subclasses to fill in
State	Change behavior when internal state changes

A.7 Composition vs Inheritance — Cheat

- **Prefer composition over inheritance** (Effective Java Item 18).
- Composition = HAS-A; Inheritance = IS-A.
- Inheritance is TIGHT coupling; composition is LOOSE.
- Fragile base class: changes to parent break subclasses.
- Composition allows multiple 'parents' via multiple components.

A.8 equals / hashCode Contract

- If `a.equals(b)` → `a.hashCode()` must equal `b.hashCode()`.
- `hashCode` consistent across calls if object unchanged.
- Equal `hashCode`s do NOT imply equal objects (collisions allowed).
- Break the contract → `HashMap` / `HashSet` silently misbehave.
- Modern shortcut: Java records / Kotlin data classes / Lombok `@Data`.

A.9 Language Gotchas Round-Up

Java:

- `==` compares references, `.equals()` compares content
- Integer cache is -128 to 127 only
- Type erasure — can't do `new T()` or `instanceof List<String>`
- Static methods can't be overridden, only hidden
- String is immutable and final

C++:

- Always virtual destructor in a polymorphic base class
- Rule of Three / Five — destructor + copy ctor + assignment come together
- Object slicing when passing by value to base type
- Non-virtual method by base pointer → NO polymorphism
- Deep copy needed if class owns raw pointers

Python:

- Mutable default arguments — the same list is reused every call
- `__x` is convention; `__x` triggers name mangling but still accessible
- Class attributes vs instance attributes distinction
- MRO decides multiple inheritance dispatch (C3 linearization)

A.10 LLD Template — 5 Steps in 45 Minutes

1. Clarify requirements (5 min) – ask, don't assume
2. Identify entities & actors (5 min) – nouns become classes
3. Define relationships (10 min) – IS-A, HAS-A, class diagram
4. Apply design patterns (10 min) – where do they fit naturally?
5. Code core classes (15 min) – skeleton, happy path

A.11 Top 10 LLD Problems Mapped

Problem	Key Patterns
Parking Lot	Factory, Strategy, Singleton, State
Elevator System	Strategy (scheduling), State, Command
ATM	State, Command, Strategy, Chain of Responsibility
Vending Machine	State machine, Strategy

Problem	Key Patterns
Chess	Strategy (piece moves), Command (undo), MVC
Splitwise	Strategy (split), Observer
LRU Cache	HashMap + Doubly Linked List; Decorator
Logger	Singleton, Chain of Responsibility, Strategy (appender)
Rate Limiter	Strategy (algo), Decorator (wrap service)
Notification System	Observer, Strategy (channel), Chain (retry)

A.12 Interview Do's and Don'ts

Do:

- Ask clarifying questions BEFORE coding
- Think out loud — interviewers grade your reasoning too
- Justify decisions in SOLID / pattern vocabulary
- Start with interface / class skeletons, fill in later
- Use polymorphism instead of if-else on type

Don't:

- Make everything static — signals you don't get OO
- Put all logic in one god class
- Go > 3 levels deep in inheritance
- Force patterns where they don't fit (patternitis)
- Fall silent — even wrong reasoning out loud beats silence

All the best for your interviews — go crush it!